ShieldVision: Advanced Gun Detection and Alerting System

Final Year Project

Session 2021-2025

A project submitted in partial fulfilment of the

NFC(IEFR) Degree

of

B.Sc. in Computer Science



School of Computing Sciences

NFC Institute of Engineering and Fertilizer Research, Faisalabad

Fall 2025

Project Detail

Type (Nature of project)	[ ] **D**evelopment [✓] **R**esearch [ ] **R&D**			
Area of specialization	Deep learning			
Project Group Members				
Sr.#	**Reg. #**	**Student Name**	**Email ID**	**Signature**
(i)	2021-UET-NFC-FD-CS-01	Areej Nadeem	nadeemareej63@gmail.com	
(ii)	2021-UET-NFC-FD-CS-02	Minha	minhaislam07@gmail.com	
(iii)	2021-UET-NFC-FD-CS-27	Saira Fatima	sairafatima@tutamail.com	

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others.

Plagiarism Free Certificate

This is to certify that, I **Minha** S/D of M. Islam, group leader of FYP under registration no 2021-UET-NFC-FD-CS-02 at Computer Science Department, NFC Institute of Engineering and Fertilizer Research Faisalabad. I declare that my FYP report is checked by my supervisor.

Date: _____ **Name of Group Leader:** Minha **Signature:** _____

Name of Supervisor: Prof. Dr. Muhammad Sultan Zia

Designation: Head of SCS

Signature: _____

HOD: Prof. Dr. Muhammad Sultan Zia

Signature: _____

APPROVAL

PROJECT SUPERVISOR

Comments: _____

_____

Name: Prof. Dr. Muhammad Sultan Zia

Date:_____ Signature:_____

PROJECT COORDINATOR

Comments: _____

_____

Date:_____ Signature:_____

HEAD OF THE DEPARTMENT

Comments: _____

_____

Date:_____ Signature:_____

Dedication

We dedicate our work and efforts to Allah, the creature of this universe who is most Bountiful and Merciful. He is indeed always with us guiding us to our destiny and the last Holy Prophet Hazrat Muhammad (P.B.U.H) the cause of the creation of this universe. Our Parents, who sacrifice for us in countless ways and our respected Teachers whose struggle and care has brought us to this stage of knowledge.

Acknowledgements

In the name of Allah, the One who is singular, perfect, and free from any uncertainty — the Most Merciful, the Most Kind. All praise belongs to Almighty Allah. May countless blessings and peace be upon His final Prophet, Hazrat Muhammad (P.B.U.H). Above all, we bow in gratitude to Allah, the Glorious, for His endless help and blessings, without which this work would never have been possible. We are deeply grateful to our university for giving us the opportunity to complete this thesis. Our heartfelt thanks go to our respected project supervisor and Head of the Computer Science Department, Prof. Dr. Muhammad Sultan Zia, for his invaluable guidance, encouragement, and thoughtful suggestions throughout this research. His support, motivation, and provision of essential resources have been vital in making this project a success. We also extend our warmest appreciation to our families, whose love, prayers, and constant support have been our greatest strength during this journey, especially throughout our engineering studies. It is because of their care and encouragement that we were able to complete this work.

Abstract

In the modern era, as security threats grow more complex and unpredictable, the demand for smart and automated monitoring systems has never been more critical. ShieldVision is a security solution powered by AI, crafted to actively identify and address gun-related threats as they occur. Utilizing advanced object detection algorithms, particularly an improved variant of the YOLO (You Only Look Once) model, the system detects firearms with remarkable precision and speed via real-time camera feeds. This improves situational awareness and allows for prompt counteractions. When a gun is detected, ShieldVision quickly creates and sends automated notifications to assigned security staff via integrated communication channels like SMS and WhatsApp. These notifications come with immediate visual proof of the danger, allowing for faster and better-informed decisions. ShieldVision provides a notable advantage in mitigating potential threats by reducing human reliance and response time before they worsen. The architecture of the system is crafted to be versatile, scalable, and suitable for various settings — including private residences, educational facilities, as well as highly secure commercial and public spaces. Its modular framework guarantees straightforward incorporation with current security systems. ShieldVision aspires to function not just as a preventive measure but also to aid digital forensics by maintaining image-based proof for every alert. It signifies progress in smart surveillance, merging computer vision, real-time communication, and AI-powered automation to safeguard individuals, assets, and property more intelligently and effectively. In the end, ShieldVision represents not merely a technological progression but a vision for safer, more intelligent, and secure environments.

Nomenclature

Keywords	Term
API	Application Programming Interface
YOLO	You Only Look Once
AI	Artificial Intelligence
SMS	Short Message Service
CCTV	Closed Circuit Television
CNN	Convolutional Neural Network
R-CNN	Regions with Convolutional Neural Network
mAP	mean with Average Percision
RPN	Region Proposal Network
SVM	Support Vector Machin
FPS	Frame Per Second
NMS	Non Maximum Suppression

Table of Contents

Project Detail ii
Plagiarism Free Certificate iii
Dedication v
Acknowledgements vi
Abstract vii
Nomenclature viii
Table of Contents ix
List of Figures xiii
List of Tables xiv
List of Equations xiv
Chapter 1: Introduction 2
1.1 Background: 2
1.2 Project Overview: 3
1.3 Objectives: 3
1.4 Scope of the Project: 3
1.5 Assumption and Constraints: 4
1.6 Significance and Benefits: 5
Chapter 2: Requirement Analysis 7
2.1 Motivation for the Project: 7
2.2 Literature Review: 7
2.2.1 Traditional Methods: 8
2.2.2 Early AI/ML-Based Systems: 9
2.2.3 YOLO Family Evolution: 10
2.2.4 Comparative Studies Benchmarking: 11
2.3 Identifying the problem: 15
2.4 Proposed Approach: 15
2.4.1 Data Preparation Phase: 15
2.4.2 Model Development Phase: 16
2.4.3 Model Evaluation: 16
2.4.4 Export Model for Deployment: 16
2.4.5 Real-Time Detection & Alert Phase: 16

2.5 Stakeholders Lists: 17
2.5.1 Project Team (Students): 17
2.5.2 Supervisor: 17
2.5.3 Security Personnel: 17
2.5.4 Cloud Service Provider (AWS): 17
2.5.5 Ethical & Legal Bodies: 18
2.5.6 Investors / Funding Agents: 19
2.5.7 Training & Onboarding Staff: 18
2.5.8 General Public: 18
2.5.9 Users Receiving Alerts (via WhatsApp/SMS): 18
Chapter 3: Implementation 21
3.1 Customization of YOLO model: 21
3.1.1 Base Architecture: YOLOv11: 21
3.1.2 Proposed Model: YOLOv11 with CBAM Integration: 27
3.2 Model training: 31
3.2.1 Data Preprocessing: 31
3.2.2 Setting Hyperparameters: 32
3.2.3 Training the Model: 32
3.2.4 Validation and Early Stopping: 32
3.3 Development Environment Setup: 33
3.3.1 Software and Tools Configuration: 33
3.4 Deployment: 34
3.4.1 Live Streaming Setup: 35
3.4.2 Larix Broadcaster (Simulated CCTV): 35
3.4.3 AWS EC2 Instance: 36
3.4.4 EC2 Instance Creation: 36
3.4.5 Connecting to EC2 via SSH: 36
3.4.6 Uploading Project Files to EC2: 37
3.4.7 Nginx with RTMP Module: 37
3.4.8 HLS Web Player: 37
3.4.9 Flow of Data: 37
3.4.10 Installing Required Libraries: 37
3.4.11 Gun Detection using customized YOLO11: 38

3.4.12 Alerting & Evidence Storage:....38
3.4.13 Google Drive Integration (PyDrive): 39
3.4.14 Environmental Security & Cleanup....39
3.5 Frontend Development: 40
Chapter 4: System Testing and Validation 42
4.1 Testing Methodology:....43
4.1.1 Types of Testing: 44
4.1.2 Test Scenarios and Use Cases:....45
4.2 Functional Testing: 46
4.3 Performance Testing: 47
4.3.1 Load Testing: 47
4.3.2 Stress Testing: 48
4.4 User Acceptance Testing: 49
4.4.1 Involvement of Beta Testers:....49
4.4.2 Feedback Collection and Analysis: 50
Chapter 5: Results and Discussion....52
5.1 Evaluation of Objectives: 52
5.2 Performance metrics: 52
5.2.1 Accuracy: 53
5.2.2 Precision: 53
5.2.3 Recall: 53
5.2.4 F1 score: 54
5.2.5 Mean Average Precision (mAP):....54
5.2.6 Comparison with Existing Methods: 54
5.2.7 Result Summary:....58
5.3 Challenges and Mitigations:....59
Chapter 6: Conclusion and Future Directions 61
6.1 Summary of Findings:....61
6.2 Achievement of Objectives: 61
6.3 Lessons Learned:....62
6.4 Future Enhancements and Upgrades: 63
6.5 Final Remarks:....63
References 65

Appendix A: Model Details and Evaluation Logs 67
A.1. Model Configuration 67
A.1.1. Dataset Overview 67
A.1.1.1. Evaluation Metrics 67
Appendix B: Administrator Manual 67
B.1. System Requirements 68
B.1.1. Installation Guide 68
B.1.1.1. Running the System 68
Appendix C: Promotional Material 69
C.1. Brochure 69
C.2. Flyer 69
C.3. Standee 69
C.4. Banner 69

List of Figures

Figure 2.1: Proposed System 19
Figure 3.1: Base architecture of YOLO11 22
Figure 3.2: C3K2 block 23
Figure 3.3: SPPF block 24
Figure 3.4: C2PSA block 26
Figure 3.5: Proposed Modified YOLO11 architecture 28
Figure 3.6: CBAM block 29
Figure 3.7:Augmentation 32
Figure 3.8: Larix broadcaster app 35
Figure 3.9: EC2 instance 36
Figure 3.10: WhatsApp Alert 39
Figure 3.11: Email Alert 39
Figure 3.12: Web interface 41
Figure 4.1: Types of testing 45
Figure 5.1: Comparison of state of the art models with ours (Bar chart) 56
Figure 5.2: Comparison of state of the art models with ours (Line chart) 56
Figure 5.3: Training results 59

List of Tables

Table 2.1: Comparative Analysis of Existing Models, Datasets and Limitations 15
Table 3.1: Training parameters.....31
Table 5.1: Performance metrics.....53
Table 5.2: Performance Metrics of the Proposed and Baseline Models 55

List of Equations

Eq. 1:Accuracy……...53
Eq. 2:Precision……...53
Eq. 3:Recall……54
Eq. 4:F1 score……54
Eq. 5:mAP……54

Chapter # 1
Introduction

Chapter 1: Introduction

ShieldVision is an advanced gun detection and alerting system that detects guns in real-time by using customized,modified state of the art YOLO object detection model and then sends alerts to the security personals along with detected frame, timestamp and location of the camera via whatsapp, email, sms and calls to ensure efficient and rapid response. Its design is flexible, scalable and deployable from home to high commercial areas where guns are prohibited. ShieldVision is a proactive solution that is built to protect lives and critical assets from real-time gun related threats.

1.1 Background:

In the past few years, the number of weapon-related incidents particularly gun-related violence has increased globally. These events have created widespread fear and placed both lives and valuable assets at risk. Crowded places such as parks, schools, markets, and most of the public areas are more likely to be exposed to such risks. Mass shootings defined as incidents involving injury or death of four or more individuals have emerged as one of the most critical challenges to law enforcement and public safety agencies worldwide (1) (2). In addition to mass shootings, crimes such as robbery and assault also contribute to the alarming rate of gun-related incidents (3).

Traditional surveillance systems, including CCTV, offer only passive monitoring and heavily rely on continuous human observation, making them prone to delays, human errors, and inefficiency in emergency situations (4). This calls for an intelligent and proactive surveillance system that not only monitors but also detects, alerts, and responds to potential threats automatically. ShieldVision is developed to fulfill this need by combining real-time object detection with automated communication technologies to create a reliable, scalable, and responsive gun detection system.

1.2 Project Overview:

ShieldVision is an intelligent surveillance system that uses advanced computer vision techniques, designed to detect firearms in real-time. The system utilizes a custom-trained YOLO-based deep learning model to identify multiple guns within a video feed, enabling fast and accurate threat recognition. It integrates with WhatsApp and email to instantly notify concerned authorities, providing a screenshot and location of the threat. The evidence is automatically uploaded to Google Drive for secure access and review. ShieldVision continuously monitors the feed and sends alerts until a human response is received, ensuring no threat is overlooked. The system is refined for real-time performance using RTMP streams and can be deployed on cloud platforms like AWS for scalability and accessibility.

1.3 Objectives:

The main objectives of the ShieldVision are:

- To develop a real-time weapon detection system that captures live footage from CCTV or surveillance cameras and processes it using an enhanced YOLO-based model to detect guns.
- To improve detection accuracy and speed, minimizing false positives and ensuring immediate identification of potential threats.
- To enable multi-object detection, allowing the system to identify multiple firearms within a single frame.
- To integrate automated alerting mechanisms via SMS and WhatsApp, sending images or video snapshots to security teams or authorities.
- To ensure 24/7 continuous monitoring with minimal human supervision.
- To make the system adaptable and deployable in diverse environments, including schools, offices, hospitals, and public spaces.

1.4 Scope of the Project:

ShieldVision is designed to:

- Detect guns in real-time using deep learning based object detection.

- Integrate with existing CCTV infrastructure.
- Send instant alerts using WhatsApp and SMS.
- Operate continuously, 24/7, in various public and private settings.

1.5 Assumption and Constraints:

Assumptions:

- In the development and implementation of the ShieldVision system, several assumptions have been made to ensure the effective functioning of the proposed solution:
- It is assumed that the video feed provided to the system is of sufficient resolution and stability to allow accurate detection.
- The trained YOLO-based model is assumed to perform reliably with the provided dataset, producing minimal false positives or negatives.
- A stable internet connection is assumed to be available at all times for sending real-time alerts via WhatsApp and email, and for uploading evidence to Google Drive.
- It is assumed that the designated authority or recipient of the alerts will monitor and respond to them in a timely manner.
- The system is designed assuming it will be deployed in environments where surveillance cameras are already installed or can be easily integrated.

Constraints:

- While designing the ShieldVision system, several constraints were identified that influence its development and performance:
- Limited access to high-performance GPUs such as A100 or V100 may affect the training and inference time of the model.
- The real-time detection accuracy may vary depending on lighting conditions, camera angles, and network latency.
- The system is currently constrained to detecting only guns; it does not support detection of other weapons or suspicious activities.

- Reliance on third-party services such as Twilio (for WhatsApp messaging) and Google Drive introduces limitations due to API rate limits, service downtime, or connectivity issues.
- The model may misclassify toy guns as real weapons or fail to differentiate between them, potentially leading to false alerts.

1.6 Significance and Benefits:

By combining AI-powered object detection with smart alerting mechanisms, ShieldVision aims to revolutionize the way public safety is managed. The system significantly improves the speed and accuracy of threat detection while reducing human error and false positives. With ShieldVision, security teams are empowered with timely information, allowing them to act swiftly and prevent potential disasters. In environments like schools, shopping malls, transportation hubs, and corporate offices, ShieldVision can serve as a critical line of defense, enhancing security without compromising convenience or cost.

Chapter # 2
Requirement Analysis

Chapter 2: Requirement Analysis

This chapter demonstrates a detailed analysis of the needs necessary for the successful development of the ShieldVision system. It begins with the motivation behind the project, emphasizing the rising concern of public safety and the need for real-time threat detection. A thorough literature review is conducted to understand existing methods and technologies, highlighting gaps and opportunities. The problem is then clearly identified, followed by the proposed approach that leverages YOLO-based deep learning and live video streaming. The chapter also includes a list of key stakeholders, such as security agencies, law enforcement bodies, educational institutions, and large public venues, who can benefit from the deployment of this system.

2.1 Motivation for the Project:

First of all, we were interested in computer vision. We wanted to build a project related to object detection. So we started search for ideas in this field and we came across to a weapons related project from which we came up with the idea of gun detection project. Furthermore, the increasing rate of gun-related incidents and mass shootings around the world has raised serious concerns regarding public safety. So, by considering this fatal threat to human life, we decided to do this project as our final year project.

2.2 Literature Review:

Security and safety have become critical concerns in today's modern world (5). Gun violence, in particular, has escalated in recent years, posing a serious threat to children, adolescents, and the general population Every year, a significant portion of the global population is affected by gun-related incidents (6) (7) creating an urgent need for advanced and proactive safety solutions This increase in violent incidents has transformed the development of intelligent surveillance from a choice into a necessity. In response to this growing demand, our research aims to develop a robust gun detection system to reduce response times and enable proactive threat mitigation.

2.2.1 Traditional Methods:

Traditional weapon detection methods, particularly metal detectors, are widely used in schools and public buildings; however, they come with significant limitations. These devices primarily exist in two forms: walk-through gates installed at fixed entry points, and handheld wands used for manual inspection. While they can detect metallic objects such as firearms, their functionality is restricted to single checkpoints and requires constant human operation. Walk-through detectors cost between 4,000 and 5,000 per unit, not including additional expenses for staff training and ongoing maintenance. Handheld devices, though cheaper, demand the presence of extra personnel, leading to long-term operational costs, where full-time guards earn between 32,000 and 45,000 annually (8). Beyond financial burdens, these systems frequently cause false alarms triggered by everyday items like keys and phones, slowing down entry and creating bottlenecks during peak hours. Moreover, they offer no visual context or real-time recording if a weapon bypasses detection, making post-incident verification impossible. Psychologically, the presence of such equipment in educational settings can feel invasive, projecting an atmosphere of distrust and transforming learning environments into high-surveillance zones.

Imaging technologies offer alternative approaches for weapon detection, as outlined comprehensively by (9). Techniques such as X-ray backscatter, millimeter-wave imaging, terahertz imaging, and ultrasound, which enable non-intrusive screening of individuals and luggage. These technologies are widely implemented in environments where is high-security like airports and correctional facilities due to their ability to detect concealed items beneath clothing or within objects efficiently. Despite their utility, these imaging systems face notable limitations. A major issue is privacy concerns, as some techniques produce detailed anatomical images that raise ethical and legal issues, particularly in civilian contexts. In addition, the effectiveness of these systems depends on skilled human operators, and performance may decline due to fatigue or interpretive errors. Moreover, these technologies are generally deployed as fixed-point solutions, which limits their mobility and prevents real-time, wide-area surveillance applications. For security and surveillance purposes and to keep an eye on unusual activities like robbery or any threat to Human life and assets, Closed Circuit Television (CCTV) cameras are used. Using CCTV cameras human supervision and effort

is needed. So, to mitigate these activities we need an AI based system to detect any harmful weapon. (5).

2.2.2 Early AI/ML-Based Systems:

Several studies explore advanced techniques for object detection and event recognition in video surveillance systems. It focuses on leveraging machine learning methods, containing Support Vector Machines (SVM), AdaBoost, and Bayesian networks, to improve the detection accuracy, efficiency and performance of surveillance systems. By analyzing human behavior, detecting objects, and understanding the context within video footage, these AI-driven techniques improve real-time monitoring and alerting capabilities. The study highlights the integration of these methods into surveillance frameworks, aiming to automate the process of recognizing suspicious activities. A significant limitation noted in the paper is the reliance on high-quality labeled data for training models, which can be a challenge in real-world applications (10).

A recent study presents an AI-driven surveillance system that combines weapon detection with facial expression analysis to improve real-time threat assessment, moving beyond traditional CCTV's role as passive evidence collectors (Singh et al., 2020). This approach addresses the limitations of earlier weapon detection methods, which often triggered false alarms in benign scenarios, such as the presence of replica weapons or in environments where weapons are common but non-threatening. In this system, a VGGNet-inspired CNN model (suspExpCNN) achieves 66accuracy in detecting high-risk facial expressions (anger, fear, or sadness), while Faster R-CNN and SSD models detect weapons with approximately 82.5accuracy. By integrating these models, the system generates alerts only when weapon presence aligns with threatening facial expressions, offering a more nuanced, context-sensitive response that mirrors human judgment. This innovative solution highlights significant progress in AI-enabled safety and incident prevention (11). This research paper shows methods or techniques for detecting guns and knives in video surveillance using Faster R-CNN (Region-based Convolutional Neural Network). It compares the effectiveness of two prominent CNN architectures, GoogleNet and SqueezeNet, within the Faster R-CNN framework for object detection tasks. The research emphasizes the significance of crucial performance metrics such as precision, recall, and mean average precision (mAP) for assessing detection accuracy. The results demonstrate that Faster

R-CNN, when paired with optimized CNN models, significantly enhances the detection of weapons in surveillance systems, improving real-time security monitoring (12).

2.2.3 YOLO Family Evolution:

The foundational YOLO (You Only Look Once) model introduced a single-pass detection framework enabling high-speed processing, beneficial for real-time security environments requiring rapid and precise firearm detection (13).

Subsequent research comparing YOLOv2, YOLOv3, and YOLOv4 shows progressive improvements in detection capabilities, with YOLOv4 achieving superior mAP, precision, and recall due to architectural enhancements like Cross-Stage Partial connections and Mish activation, solidifying its effectiveness for real-time weapon detection (14) (15).

The research presents a Weapon Detection System using YOLOv5 for real-time detection and Mask R-CNN for instance-level segmentation. YOLOv5 leverages ResNet-101 for feature extraction, coupled with a Region Proposal Network (RPN) to create and refine important regions. Mask R-CNN is applied for pixel-level segmentation, improving detection precision. The results demonstrate that YOLOv5 outperforms earlier models like YOLOv3, YOLOv4, and Faster R CNN in terms of both speed and accuracy, making it highly suitable for real-world surveillance applications (16).

This research presents a deep learning model for real-time fire and gun detection, leveraging the YOLOv5 algorithm for accurate and fast object detection. The primary objective is to detect emergencies such as fires, industrial explosions, and gun violence, enabling timely interventions. YOLOv5 was chosen for its superior performance, offering higher frames per second (FPS) and enhanced accuracy compared to previous YOLO versions. The model processes video frames in real time, identifying objects such as fire and guns by drawing bounding boxes around detected items and displaying their respective names and prediction values. The study demonstrates that YOLOv5 outperforms other models like Faster R-CNN and SSD in both speed and accuracy. The proposed system can significantly reduce potential harm and property damage by providing early warnings for fire and gun-related incidents (17).

Recent advancements in real-time weapon detection have utilized deep learning models, specifically object detection algorithms, to improve security protocols in public areas. A study showcases an approach that utilizes the YOLOv8 (You Only Look Once) model, recognized for its rapid processing

and impressive accuracy in real-time image analysis. The model employs a strong framework that encompasses a backbone for extracting features, a neck for integrating features, and a head for generating final predictions, all vital for real-time use in security systems. The research highlights the significance of loss functions, including localization, confidence, and classification losses, in the training phase to enhance the model's accuracy. In addition, metrics such as precision, recall, F1-score, and mean average precision (mAP) are employed to evaluate model performance. These metrics are essential for comprehending the model's capability to accurately identify weapons amid differing degrees of overlap between predicted and actual bounding boxes. The actual application of the paper entails real-time inference, utilizing the trained YOLOv8 model to analyze video frames, forecast weapon locations, and implement Non-Maximum Suppression (NMS) to eliminate redundant predictions. The findings, featuring precision-recall curves and confusion matrices, offer important perspectives on the model's advantages and drawbacks. This research is especially applicable for initiatives focused on implementing AI-based security solutions, as it provides an extensive approach for developing and accessing a real-time weapon detection system. YOLOv8's efficacy in practical situations positions it as a favorable option for security use, and its methodology can guide upcoming efforts to enhance detection precision and real-time capabilities (18).

2.2.4 Comparative Studies Benchmarking:

Fernandez-Carrobles (12) proposed two models for detecting knives and handguns: one using Faster R-CNN with a GoogleNet backbone and the other with a SqueezeNet backbone. The study examines the detection of handguns and knives, but these were addressed as distinct tasks, leading to two separate training phases for each model: one for handguns and another for knives. To detect handguns, the researchers utilized a dataset from an earlier study and employed data augmentation methods to enhance realism, resulting in 15,000 training images in total. To detect knives, researchers took knife images from the MS-COCO dataset and utilized data augmentation methods once more, resulting in a total of 12978 images. The test datasets were created by combining prior research and recognized open-source datasets like MS-COCO and Open Images Dataset V4. The model based on SqueezeNet achieved the highest Average Precision (AP) of the two models at 0.5 Intersection over Union (IoU) in the gun detection task, scoring 85.44.

Vallez (19) created a Deep Learning approach utilizing a Faster R-CNN framework combined with an autoencoder to address the issue of elevated false positive (FP) rates. The aim of the autoencoder was to further examine positive detections and ultimately determine if they were true positives (TP) or false positives (FP) by analyzing the reconstruction error. Because the images utilized to train the encoder were FP, the reduced errors signified FP and distinguished them from TP based on a threshold. Additionally, the authors produced a synthetic dataset of 3000 samples utilizing Unreal Engine 4, which served as inputs for the model. The quantity of false positives decreased by 37.9% with the assistance of the autoencoder, while the detection capability stayed consistent when compared to just the Faster R-CNN. The researchers also observed that detection capability and the frequency of false positives fluctuated. Nonetheless, the autoencoder preserved stability between these values by allowing both of them to remain unaffected for the initial range of thresholds.

Loganathan (20) utilized a Faster R-CNN with an InceptionV2 backbone for detecting firearms in surveillance footage. The model underwent pretraining using the MS-COCO dataset. The authors created a dataset by manually annotating images sourced from different platforms, including public databases and Google images, yielding 3908 training samples and more than 100 authentic surveillance samples for testing. The model achieved a testing accuracy of 89.4.

Lim (20) introduces an enhanced deep multi-level feature pyramid network that reduces comprehension issues for firearms from an unconventional perspective. The authors demonstrate that current datasets for training do not accurately reflect the reality of video surveillance images, so to solve this issue, they created their own dataset. This dataset includes 603 images sourced from Flickr and 2857 from videos recorded by the authors in authentic settings, all accounting for lighting conditions, camera angles, and background context. Moreover, research indicates that the generated dataset exhibited consistent classifications of object size, count, and brightness across all images when compared to the Granada(REF) and UFC Crime (REF) datasets. The chosen model for detection was the MLMSOD (Multi-level Multi-scale Object Detector), enhanced with a new Feature Fusion Module (FFMv3) and the integration of Generalized Intersection Over Union (GIoU) Loss along with Focal Loss. All architectures were trained on a combination of the Granada and authors' datasets to show that the proposed architecture

outperforms the state-of-the-art. The models evaluated included Faster R-CNN with FPN, Mask R-CNN, YOLOv3, CenterNet, RetinaNet with NAS-FPN, Dynamic R-CNN, and EfficientDet-D7x, with the suggested solution attaining an average precision boost of 43.02% compared to the baseline and a 49.4% enhancement in mAP over all other contenders in the refined version. Additionally, the authors incorporated the Adaptive Surveillance Image Partitioning (ASIP) in their model, which divides the image into various sizes with slight overlaps and carries out detection for each segment. This technique enhanced the classification scores, resulting in a minimum accuracy of 80% and a maximum of 50% at various distances (3 to 8 m from the camera to the gun).Eventually, it is come to end that the model achieves 12 FPS in inference and is, therefore, fit for real-time detection.

Hashmi (14) performed a comparative study on two object detection algorithms used for weapon detection: YOLOv3 and YOLOv4. For the purpose of comparison, the authors assembled a dataset consisting of 7800 weapon images obtained from various sources including Google images, CCTV footage, and films. The evaluation was made using precision, recall, f1score, and mAP metrics, where YOLOv4 surpassed YOLOv3 on all metrics, achieving an mAP score of 84.85.

Kong (20) created a novel single-stage detection model for identifying military targets. The suggested model is called YOLO-G, and its design is derived from YOLOv3 with three enhancements included. To enhance the original YOLOv3 baseline, the authors substituted the DarkNet-53 backbone with a GhostNet backbone to boost performance. The second enhancement involved adding a coordinate attention mechanism to enhance detection performance. The last enhancement involved employing an alternative loss function that incorporated Distance-IoU (DIoU) alongside Focal Loss. A new dataset was generated for performance evaluation, yielding 5000 samples collected from the Bing and Google search engines. Two experiments were performed: an ablation study to assess the advantages of each enhancement against the YOLOv3 baseline and a comparative study to evaluate the enhanced model's performance relative to other leading architectures. In the ablation study, YOLO-G attained a 0.91 and 1 boost in mAP with each enhancement, respectively, along with a 25.9 FPS rise. The suggested model was evaluated against four other models in the comparative experiment: Faster R-CNN, SSD, YOLOv3, and YOLOv5s. Results indicated that YOLO-G outperforms in all metrics except detection speed, where it is surpassed only by the newest state-of-the-art one-stage detection model, YOLOv5s, which achieved nearly double the FPS.

The paper "Real-Time Weapon Detection Using YOLOv8 for Enhanced Safety" by Ayush Thakur, Akshat Shrivastav, Rohan Sharma, Triyank Kumar, and Kabir Puri (18) explores the use of YOLOv8 for real-time weapon detection. The authors employ YOLOv8's CSPDarknet backbone to enhance detection accuracy and speed. Using a diverse dataset, including publicly available data, the model achieves an accuracy of 92 with key performance metrics including a precision of 73 recall of 80 and an F1-Score of 85. Despite its success, limitations such as overfitting, false positives, and variable detection speeds were identified, emphasizing the need for multi-modal integration in future work.

Model	**Backbone**	**Dataset**	**Limitation**
Faster R-CNN [2019]	Squeeze-Net (12)	15,000 images MS-COCO, Open Images	Handgun detection performance may degrade in complex environments. Dataset not extensive for complex scenarios.
Faster R- CNN + Autoencoder [2019]	Not specified(21)	3000 images and Unreal Engine 4-generated synthetic data	Synthetic data may not perfectly match real world conditions.
Faster R-CNN [2019]	InceptionV2 (22)	3908 images and Google images, public databases	Lower performance in detecting complex or occluded weapons.
Multi-level Multi- scale Object Detector[2021]	Multi-level Multi- scale Object Detector (MLMS OD) (22)	3459 images and Custom Dataset	The model performs less effectively at longer distances, with reduced accuracy (50 at long distances).
YOLOv3, YOLOv4 [2021]	Darknet-53,CSP Darknet53 (14)	7800 images and Weapons dataset from Google images, CCTV, movies	Lower mAP compared to YOLOv4 ,None high- lighted (better performance on all metrics)

YOLO-G [2022]	GhostNet(20)	5000 images and Custom Dataset (Bing Google images)	Speed (FPS) is lower than YOLOv5s; Less detection speed for real time applications
YOLOv8 [2024]	CSPDarknet (18)	Diverse dataset and publicly available datasets	Limitations include overfitting, false positives, detection speed variability, and lack of multi-modal integration.

Table 2.1: Comparative Analysis of Existing Models, Datasets and Limitations

2.3 Identifying the problem:

Despite advancements in surveillance technology, delayed threat detection remains a critical challenge in security systems. Traditional CCTV monitoring relies heavily on human operators, leading to slow response times, human fatigue-induced errors, and inability to recognize and identify armed high-risk threats in real-time.

In high-risk environments where security is highly needed like schools, banks, and public spaces gun-related violence escalates rapidly, often before security teams can react. Manual monitoring is mostly not efficient, and existing automated systems deficit accuracy, speed, and real-time alert mechanisms, putting lives and assets at risk.

2.4 Proposed Approach:

The proposed approach outlines the step-by-step process for developing, training, evaluating, and deploying a real-time gun detection and alert system using the modified YOLOv11model. It is structured into five main phases: data preparation, model development, model evaluation, deployment, and real-time detection with alert generation.

2.4.1 Data Preparation Phase:

- **Data Collection:** The first step is to collect a large and diverse dataset consisting of images or videos containing guns and non-gun objects for training.

- **Dataset Structure:** The collected dataset is organized into a structured format suitable for training, typically containing images and corresponding annotation files.
- **Preprocessing:** The images and annotations are preprocessed to enhance data quality. This may include resizing, format conversion, or cleaning.

2.4.2 Model Development Phase:

Model Training (YOLOv11): The preprocessed dataset is used to train the custom YOLOv11 model. The YOLOv11 model consists of three main components:

- **Backbone:** Extracts features from the input images.
- **Neck:** Combines multi-scale features for better object detection.
- **Head:** Generates final detection outputs including bounding boxes and class scores.

2.4.3 Model Evaluation:

After training, the model is evaluated using test data to check its performance, accuracy, and robustness.

2.4.4 Export Model for Deployment:

Once the model performs satisfactorily, it is exported and prepared for deployment in the real-time system.

2.4.5 Real-Time Detection & Alert Phase:

- **Live Video Feed:**

The deployed system connects to live video feeds from surveillance cameras.

- **Frame Extraction:**

Frame extraction means to divide the video into continuous stream of images (frames). Individual frames are extracted from the continuous video stream for analysis because model cannot process a whole video.

- **Preprocessing:**

Extracted frames are preprocessed to match the input requirements of the trained YOLOv11 model.

- **Proposed modified YOLO11 Inference:**

The YOLO11 model processes the frames to detect the presence of guns or other threats.

- **Threat Verification:**

If a threat (gun) is detected, the system verifies the threat. If no threat is detected, the system continues monitoring.

- **Alert Generation:**

If a threat is verified, an immediate alert is generated.

- **Notification (WhatsApp message, SMS and Email):**

The alert is sent to the concerned authorities or responsible individuals via SMS, WhatsApp and email ensuring quick response.

2.5 Stakeholders Lists:

The following stakeholders are involved in ShieldVision project,

2.5.1 Project Team (Students):

Responsible for the design, development, and successful implementation of the ShieldVision system.

2.5.2 Supervisor:

Provides academic guidance, monitors project progress, and ensures it meets educational standards.

2.5.3 Security Personnel:

End users of the system who respond to real-time gun detection alerts to ensure safety.

2.5.4 Cloud Service Provider (AWS):

Hosts the deployed system and provides computing resources for real-time processing and storage.

2.5.5 Ethical & Legal Bodies:

Ensure that the system complies with data privacy, surveillance regulations, and ethical guidelines.

2.5.6 Investors / Funding Agents:

Provide financial support for system development, deployment, and potential commercialization.

2.5.7 Training & Onboarding Staff:

Responsible for educating users and operators on how to effectively use the ShieldVision system.

2.5.8 General Public:

Indirect beneficiaries of the system whose safety is enhanced through proactive threat detection.

2.5.9 Users Receiving Alerts (via WhatsApp/SMS):

Could be individuals like homeowners, building managers, or institutional heads who are alerted in case of detection.

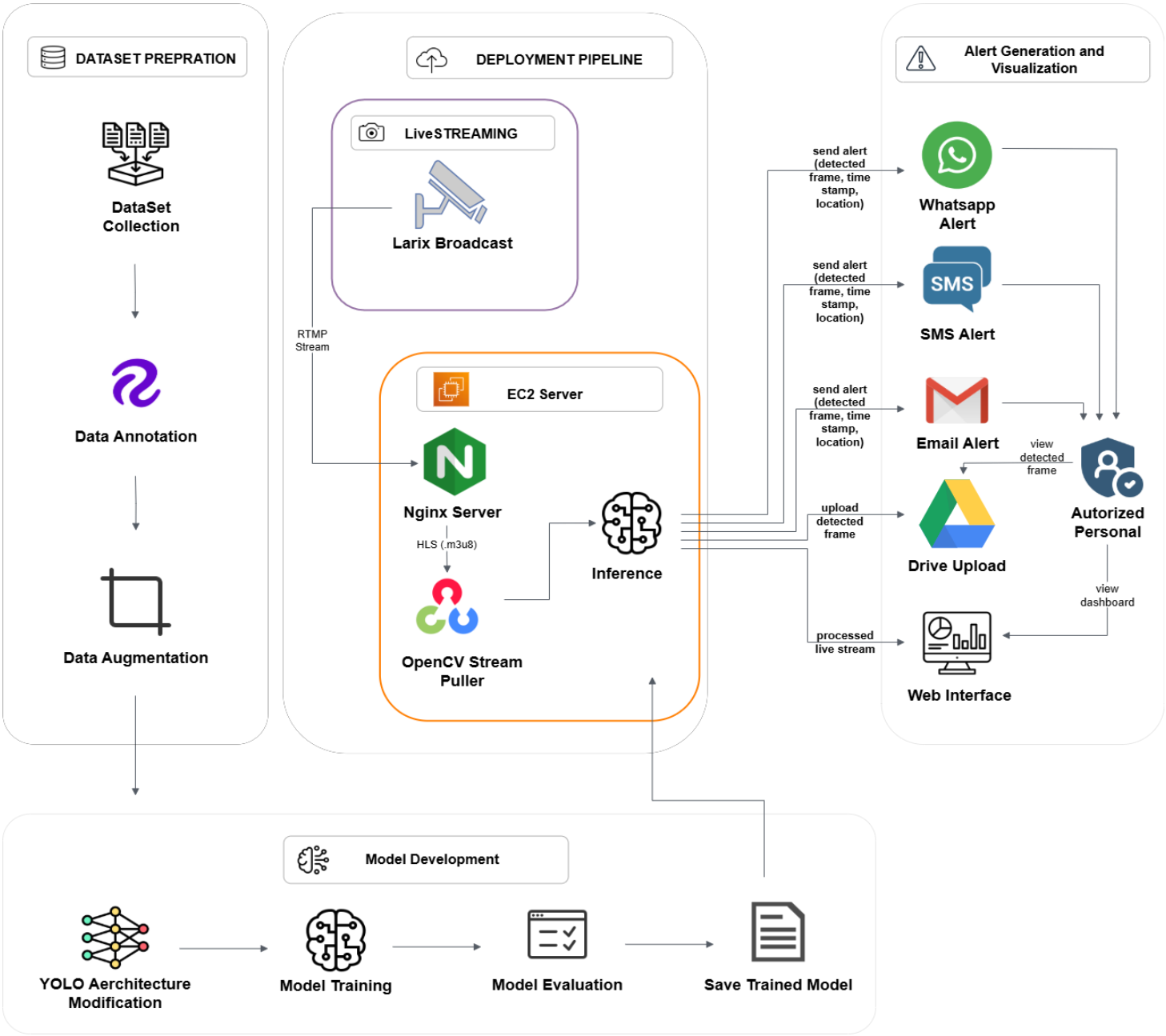


Figure 2.1: Proposed System

Chapter # 3
Implementation

Chapter 3: Implementation

The implementation of ShieldVision was carried out using Python and OpenCV, integrating an enhanced YOLOv11 model for real-time gun detection. A live RTMP stream from CCTV is sent to an AWS EC2 instance, where each frame is processed by the inference engine. If a gun is detected, the system draws a bounding box, saves the image locally, and deletes any previous detection files. The image is then uploaded to Google Drive, and an alert is sent via SMS or WhatsApp along with the IP-based location link. The detection and alerting processes run asynchronously to ensure speed and responsiveness. The entire pipeline is optimized for low latency and high accuracy in real-world surveillance scenarios.

3.1 Customization of YOLO model:

To enhance the accuracy and efficiency of real-time firearm detection, the YOLOv11 architecture was customized and further improved with attention mechanisms. The customization process includes a detailed study of the YOLOv11 base architecture, followed by the integration of the Convolutional Block Attention Module (CBAM). This approach not only leverages YOLOv11's advanced modules but also adds refined attention capabilities, ensuring better detection of small, occluded, or complex objects in surveillance environments.

3.1.1 Base Architecture: YOLOv11:

YOLOv11 (You Only Look Once version 11) is the most recent evolution of the YOLO series, developed by Ultralytics in 2024. It continues the legacy of real-time object detection models with a focus on both speed and accuracy. Unlike traditional object detection methods that rely on two-stage pipelines (like R-CNN), YOLOv11 maintains a single-shot approach, where the entire image is processed in a single forward pass to predict bounding boxes and class probabilities.

YOLOv11 introduces several architectural innovations including the **C3K2**, **SPFF**, and **C2PSA** blocks. These modules improve feature extraction, enable attention-aware learning, and enhance multi-scale detection all crucial for applications like real-time surveillance and weapon detection.

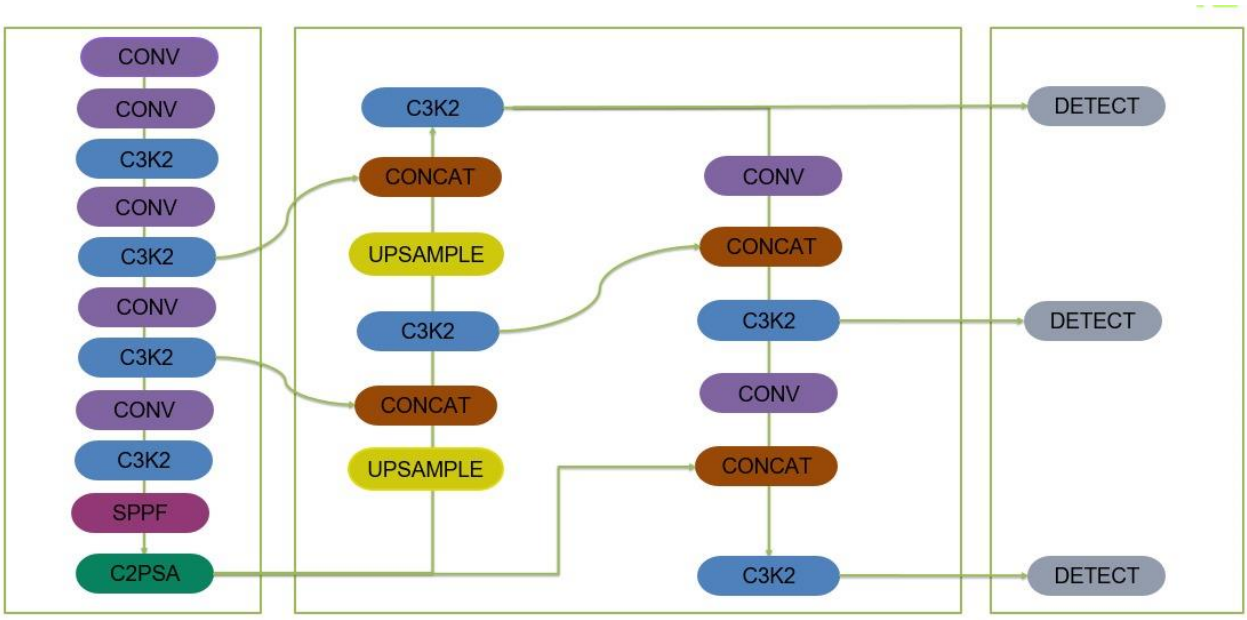


Figure 3.1: Base architecture of YOLO11

YOLOv11 comprises the following primary components:

- **Backbone:** Extracts features from raw input images using convolutional operations and specialized modules.
- **Neck:** Aggregates and fuses multi-scale feature maps for refined representation.
- **Head:** Predicts bounding boxes, class scores, and objectness confidence values.

Each block includes unique modules that contribute to the model's overall performance.

The **Backbone** is responsible for transforming raw input images into informative multiscale feature maps using a stack of convolutional layers.

Convolutional Block is named as Conv Block which process the given c,h,w passing through a 2D convolutional layer following with a 2D Batch Normalization layer at last with a SiLU Activation Function.

C3K2 Block is one of the key architectural innovations introduced in YOLOv11 is the C3K2 block, which plays a central role in the feature extraction process within the backbone. This block is a refined version of the C2f module from YOLOv8 and is specifically designed to balance computational efficiency and feature representation capability.

The C3K2 block operates using multiple smaller 3×3 convolutional kernels, which are more computationally efficient compared to larger kernels. These allow the network to extract localized patterns while keeping the model lightweight and suitable for real-time inference.

The C3K2 structure is composed of two core components:

- C3K Block begins with a convolutional layer followed by a sequence of n bottleneck layers. The outputs of these layers are concatenated and passed through a final convolution. Unlike the C2f block, no channel splitting occurs within C3K, ensuring unified feature processing.
- C3K2 Block begins with an initial convolution, followed by multiple C3K blocks. The output of the final C3K block is concatenated with the output of the first convolution. This concatenated feature map is then processed through a final convolution layer to complete the module.

This architectural design enhances feature reuse and strengthens the information flow across stages, thereby improving the model's overall accuracy. Additionally, it reduces the parameter count compared to earlier blocks, such as C2f, making it more efficient for deployment on edge devices.

By maintaining the benefits of the Cross Stage Partial (CSP) strategy while improving its internal structure, the C3K2 block significantly contributes to YOLOv11's high performance in object detection tasks, especially where both speed and precision are critical.

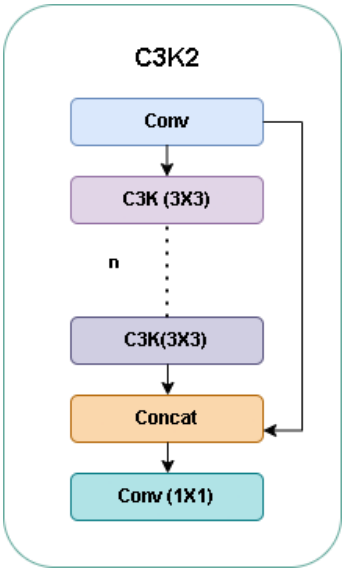


Figure 3.2: C3K2 block

The **Neck** combines multi-scale feature maps rom the backbone and transmits them to the head for prediction. It plays a crucial role in aggregating contextual information.

The **Spatial Pyramid Pooling Fast (SPFF)** module is a critical component of the neck in YOLOv11. It enhances the model's ability to detect objects of varying sizes—particularly small objects—by aggregating multi-scale contextual information without significantly increasing computational cost. Unlike traditional pooling mechanisms, the SPFF module employs multiple max-pooling operations with different kernel sizes (e.g., 5 × 5, 9 × 9, 13 × 13). These parallel pooling layers extract features from different receptive fields, capturing both fine-grained local features and broader global context. The resulting feature maps are concatenated and passed through a convolutional layer to fuse the information.

- SPPF is inspired by the original Spatial Pyramid Pooling (SPP) but optimized for real-time performance.
- It enables the network to retain high detection accuracy for small and distant objects by combining low- and high-resolution cues.
- The design is lightweight and ensures that YOLOv11 maintains its hallmark real-time inference speed.

By incorporating SPPF in the neck, YOLOv11 achieves a robust fusion of features from different spatial scales, contributing to improved object detection across diverse scenarios. Its ability to enrich feature maps without sacrificing efficiency makes it a vital component in the YOLOv11 architecture.

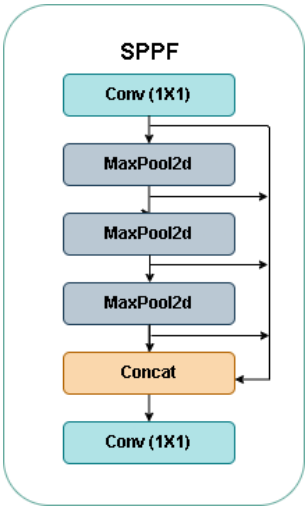


Figure 3.3: SPPF block

C2PSA (Cross Stage Partial with Position-Sensitive Attention) Block which is an attention mechanism, is another significant innovation in YOLOv11's architecture block. This module introduces a lightweight attention mechanism designed to selectively focus on the most informative spatial regions within a feature map. Its primary objective is to enhance the model's representational power, especially for small or partially occluded objects, without incurring substantial computational cost.

The C2PSA block is composed of two core components:

- A 1×1 convolutional layer that first processes the input feature map.
- The processed map is then split into two branches. Each branch is passed through a Partial Spatial Attention (PSA) block. The PSA block internally includes a multi-head attention mechanism where the input is transformed into query, key, and value (QKV) representations. Attention scores are computed and applied to enhance the feature representation. The PSA module also includes a pair of convolutional blocks and feed-forward layers to refine the attended features. After sequential application of PSA on both branches, the outputs are concatenated along the channel dimension and finally passed through another 1×1 convolution layer to fuse the information.
- The combination of attention and convolution in PSA allows the network to learn spatial dependencies more effectively.
- The block enables parallel branch processing, promoting a richer feature set while maintaining low computational overhead.

Unlike traditional attention mechanisms, PSA is optimized for real-time object detection tasks, making it well-suited for YOLOv11's design goals.

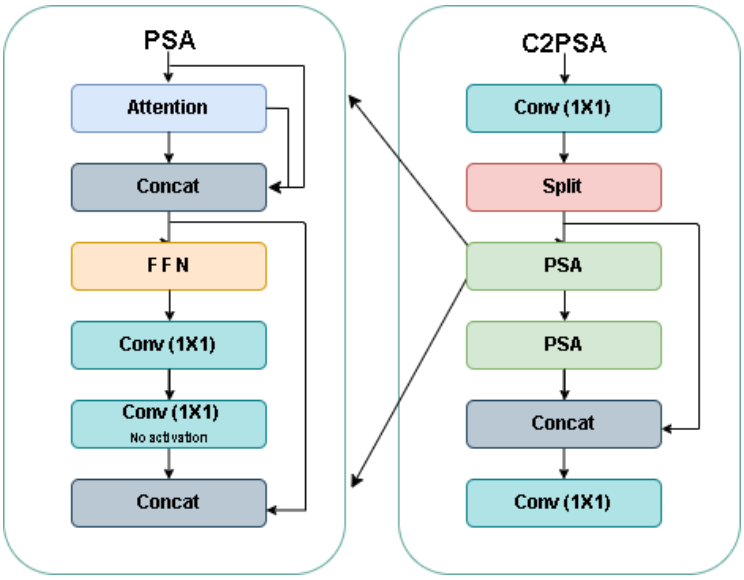


Figure 3.4: C2PSA block

Functionally, the C2PSA block is positioned after the SPFF module in the neck of the architecture. It receives multi-scale features (e.g., 20 × 20 × 1024) and applies spatial attention to emphasize important regions. The attended features are then progressively upsampled and concatenated with medium- and high-level feature maps (e.g., 40 × 40 × 512 and 80 × 80 × 256), ensuring effective fusion across scales. These fused features are passed to subsequent C3K2 blocks for additional processing before reaching the detection head. Overall, the C2PSA block plays a critical role in enhancing spatial awareness within the model, thereby contributing to YOLOv11's superior performance in complex scenes involving varied object scales and occlusions.

The detection **Head** in YOLOv11 is designed to perform object detection across multiple spatial resolutions. This component is responsible for interpreting the refined feature maps generated by the backbone and neck and converting them into precise bounding box predictions, objectness scores, and class probabilities. Consistent with prior YOLO versions, YOLOv11 utilizes a multi-scale prediction strategy, which involves processing three different feature maps corresponding to various resolutions:

- P3: High-resolution feature map (80 × 80), used for detecting small objects.
- P4: Mid-resolution feature map (40 × 40), used for detecting medium-sized objects.
- P5: Low-resolution feature map (20 × 20), used for detecting large objects.

Each of these prediction layers is responsible for producing detection outputs at its respective scale. The use of multi-scale outputs ensures that:

- Small and densely packed objects are detected using high-resolution features (P3),
- Medium and large objects benefit from coarser, semantically rich features (P4 and P5),
- The model achieves a robust balance between localization accuracy and detection efficiency across various object sizes.

Each head outputs a tensor containing bounding box coordinates (x, y, w, h), objectness confidence, and class scores. These predictions are aggregated and post-processed (e.g., via Non-Maximum Suppression) to produce the final detection results.

In summary, the multi-scale detection head in YOLOv11 plays a crucial role in delivering high performance across diverse object detection tasks, ensuring both fine-grained precision and scalable inference.

3.1.2 Proposed Model: YOLOv11 with CBAM Integration:

The proposed system builds upon the YOLOv11 architecture, a state-of-the-art object detection model known for its balance between speed and accuracy. While YOLOv11 introduces C3K2, SPFF, and C2PSA modules to enhance feature representation and attention, further improvements are achieved by integrating the Convolutional Block Attention Module (CBAM) into the model. CBAM applies channel and spatial attention sequentially, enabling the network to focus more effectively on relevant features such as weapon edges, shapes, and contours, while suppressing irrelevant background noise. By adding CBAM after selected backbone or neck layers, the model benefits from enhanced context awareness, which is especially crucial for accurate weapon detection in complex surveillance environments. This integration aims to improve detection precision without significantly compromising the model's inference speed.

To further enhance the feature extraction and attention mechanisms in the baseline YOLOv11 model, the **Convolutional Block Attention Module (CBAM)** is incorporated at various stages of the backbone and head. CBAM sequentially applies channel attention followed by spatial attention, allowing the model to selectively emphasize informative features and suppress irrelevant regions.

The CBAM (Convolutional Block Attention Module) is strategically integrated at multiple stages of the network to enhance feature representation at various semantic levels. In the

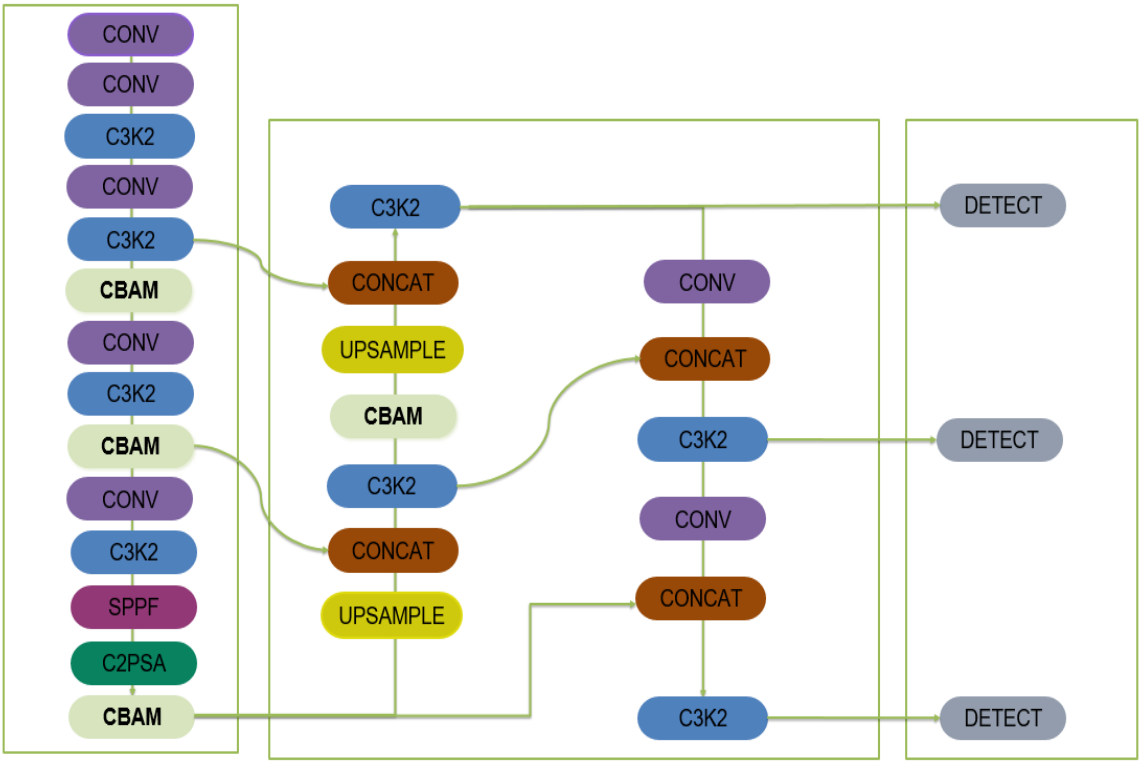


Figure 3.5: Proposed Modified YOLO11 architecture

backbone, CBAM is first applied at **Layer 5**, right after the C3K2 block at the **P3 level**, to emphasize low-level semantic features. It is then used again at **Layer 8**, following the C3K2 block at the **P4 level**, where it helps refine mid-level feature representations. At a deeper stage, CBAM is incorporated at **Layer 13**, after the C2PSA block at the **P5 level**, focusing on high-level semantic information. Additionally, in the **head** of the network, CBAM is inserted at **Layer 17**, specifically after the fusion of upsampled features and backbone outputs from **Layer 8**, to guide attention during decoding and feature fusion at the **P4 stage**.

These insertions improve attention at different pyramid scales (P3, P4, P5), aligning with the multi-scale detection strategy used in YOLO. Especially for weapon detection, where guns may be small and partially occluded, CBAM helps the model focus on discriminative parts, improving both precision and recall. The lightweight nature of CBAM ensures that computational overhead is minimal, preserving YOLOv11's real-time capability.

The **Convolutional Block Attention Module (CBAM)** is a lightweight attention mechanism that improves the representational power of convolutional neural networks by refining intermediate feature maps using both channel and spatial attention mechanisms. It sequentially applies:

1. Channel Attention Module (CAM)

2. Spatial Attention Module (SAM)

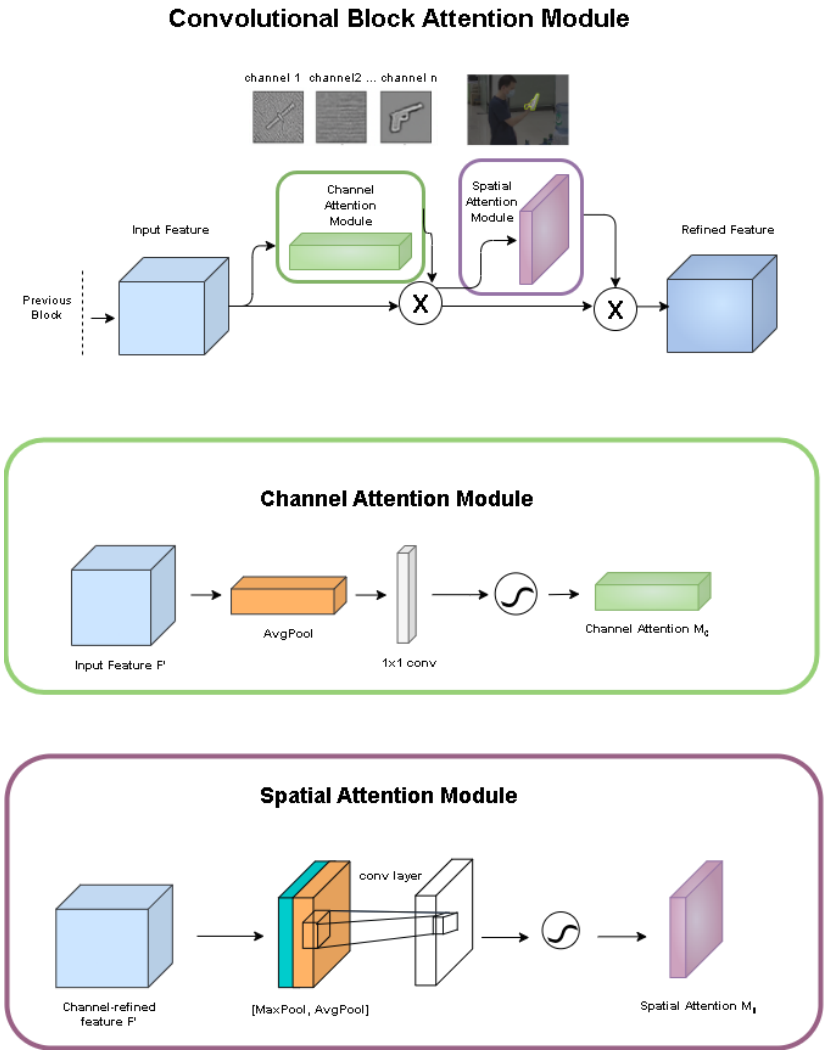


Figure 3.6: CBAM block

Given an intermediate feature map **F ∈ RC×H×W** as input, CBAM sequentially infers a 1D channel attention map **Mc ∈ RC×1×1** and a 2D spatial attention map Ms ∈ R1×H×W. The overall attention process can be summarized as:

F′ = Mc(F) ⊗ F, F″ = Ms(F′) ⊗ F′,

where ⊗ denotes element-wise multiplication. During multiplication, the attention values are broadcasted accordingly: channel attention values are broadcasted along the spatial dimension, and vice versa. F'' is the final refined output.

The goal of the **Channel Attention** mechanism is to determine what features are important. Each channel in a feature map corresponds to a learned filter that responds to certain types of features (e.g., textures, edges, object parts). However, not all channels contribute equally. The channel attention mechanism enhances the important channels and suppresses less relevant ones. To compute the channel attention map, **$M_c \in RC\times1\times1$**, the module performs the following steps:

- Apply global average pooling to compress the spatial dimensions:
  $F_{avg} = AvgPool(F)$
- Apply a 1 × 1 convolution (equivalent to a fully connected layer)
  $M_c(F) = \sigma(W_c \cdot F_{avg})$
- Multiply $M_c(F)$ with the original input F via element-wise multiplication (broad-casted across spatial dimensions):
  $F' = M_c(F) \cdot F$
- Where W_c denotes the convolutional weights and σ is the sigmoid activation function.
- The result F ' is a channel-refined feature map.

While channel attention emphasizes which features are important, **Spatial Attention** focuses on where these features are located. It identifies significant spatial regions within the feature map, which is critical for tasks like object detection and localization. To compute the spatial attention map $M_s \in R1\times H\times W$, the following steps are performed:

- Apply average pooling and max pooling along the channel axis to generate two spatial context descriptors:
  $F_{avg_s} = AvgPool_{chan}(F')$, $F_{max_s} = MaxPool_{chan}(F')$
- Concatenate the descriptors and apply a convolution with a 7 × 7 kernel:
  $M_s(F') = \sigma(f_{7\times7}([F_{avgs}; F_{max_s}]))$
- Multiply the spatial attention map with the input feature map:
  $F'' = M_s(F') \cdot F'$

Here, f 7×7 represents the convolutional operation with a kernel size of 7×7, and σ is the sigmoid activation. The output F'' is the final refined feature map containing enhanced spatial and channel information.

The overall CBAM attention process can be summarized as:

$$F' = Mc(F) \cdot F$$

$$F'' = Ms(F') \cdot F'$$

This sequential combination of channel and spatial attention enables the network to adaptively focus on informative features, improving detection accuracy while maintaining computational efficiency.

3.2 Model training:

For ShieldVision, several crucial steps would be taken in account to train the AI model. We train our modified model on our customized dataset. Our goal would be the achievement of accuracy and reliability in gun detection. We did not rely on the default hyperparameters. We will also reduce the false positives and negatives by adjusting the hyperparameters (learning rate, batch size, number of epochs, image size, optimizer, etc) for accurate and precise gun detection. Here we will take a look at the explanation of each phase in training process:

Model	**Limitation**
Epochs	483(Best model at 433)
GPU Used	Nvidia GeForce RTX 2080 Ti(11 GB)
Model Size	5.7 MB

Table 3.1: Training parameters

3.2.1 Data Preprocessing:

First the data preprocessing will take place. In this step Images are resized and normalize. Resizing means that the height and width if the images will be changed and by normalization we will get images whose pixel values are in a consistent range. The augmentation of data will also

occur. In this, we rotate, flip or adjust the brightness, etc of the image so that the model can have a more generalize view of images in different environments.



Figure 3.7:Augmentation

3.2.2 Setting Hyperparameters:

Then we will set the hyperparameters according to our requirements. These parameters include learning rate, batch size and IoU threshold. Learning rate tells that how fast the model us being trained. Batch size is the total number of images per batch and IoU (intersection over union) threshold keeps the balance in training.

3.2.3 Training the Model:

Then, the model will be trained by feeding the preprocessed images and annotation in YOLO format to the model. We will also be using loss functions which will help the model to be trained more accurately. It is trained by multiple epochs.

3.2.4 Validation and Early Stopping:

Then, the model will be validated by utilizing the validation set of images. This will prevent the risk of overfitting. Early stopping will also be applied so that it ceases the training process if there is no improvement invalidation accuracy.

3.3 Development Environment Setup:

The development environment was carefully designed to support the requirements of real-time firearm detection, alert generation, and cloud-based evidence storage. Both software and hardware components were configured to ensure smooth experimentation, model training, and deployment.

3.3.1 Software and Tools Configuration:

- **Programming Language: Python**

  Python was chosen as the primary programming language due to its simplicity and wide range of libraries suited for machine learning and computer vision tasks. It was used to customize and train the YOLO model, handle real-time video frame capture, and trigger the alert mechanism.

- **Framework: PyTorch**

  PyTorch, an open-source deep learning framework developed by Meta, was utilized to implement and train the customized YOLO model. Its dynamic computation graph and flexibility made it ideal for experimenting with architectural enhancements and optimizing real-time object detection performance in ShieldVision.

- **Computer Vision Library: OpenCV**

  OpenCV was used for real-time video processing, including capturing frames from the camera, preprocessing the input, and drawing bounding boxes around detected firearms. It also facilitated image conversion and frame analysis.

- **Object Detection Model: YOLO (You Only Look Once)**

  YOLO was selected for object detection due to its capability to detect and classify multiple objects in a single pass. Its high speed and accuracy made it ideal for real-time surveillance applications. The model was further customized to enhance firearm detection performance.

- **Communication API: WhatsApp Bussiness API**

  The WhatsApp Business API enables secure, real-time communication with users, allowing automated alerts, notifications, and customer engagement on the world's most widely used messaging platform.

- **Hardware: NVIDIA RTX 2080 Ti**

  The training of the customized YOLO model was performed using the NVIDIA RTX 2080 Ti GPU. This high-performance graphics card significantly accelerated the training and inference processes, ensuring low latency and fast model convergence.

- **Google Drive API:**

  Used to upload and store evidence images securely on the cloud, making them easily accessible through alert messages.

- **NumPy:**

  Utilized for numerical operations, particularly during image data handling and preprocessing tasks.

- **Requests Library:**

  Handles HTTP requests such as uploading files and sending data to APIs like Google Drive and Twilio.

- **AWS EC2:**

  A cloud-based virtual server used to deploy the detection system remotely, enabling 24/7 real-time monitoring and scalability.

- **Larix Broadcaster App:**

  A mobile application used to capture and stream live video from a smartphone camera to the EC2 server using the RTMP protocol.

3.4 Deployment:

This chapter presents the deployment strategy for ShieldVision — an AI-powered gun detection and alerting system. The project integrates real-time camera streaming, gun detection using a custom-trained YOLOv11 model, and automatic alert generation via WhatsApp, Email, and

Google Drive. A Flask-based web interface enables end-users to monitor live detection events from any browser.

3.4.1 Live Streaming Setup:

This setup enables real-time video streaming from a mobile device to a cloud server and then to a web browser. The mobile device uses the Larix Broadcaster application to send a live video feed to an AWS EC2 instance, where Nginx with the RTMP module processes the stream. The stream is converted into the HLS (HTTP Live Streaming) format, allowing it to be viewed on any modern web browser.

3.4.2 Larix Broadcaster (Simulated CCTV):

While Larix is primarily designed for mobile streaming applications, it served as a temporary simulation of a CCTV camera feed. This allowed for the testing and real-time ingestion of live video in a flexible and mobile manner.

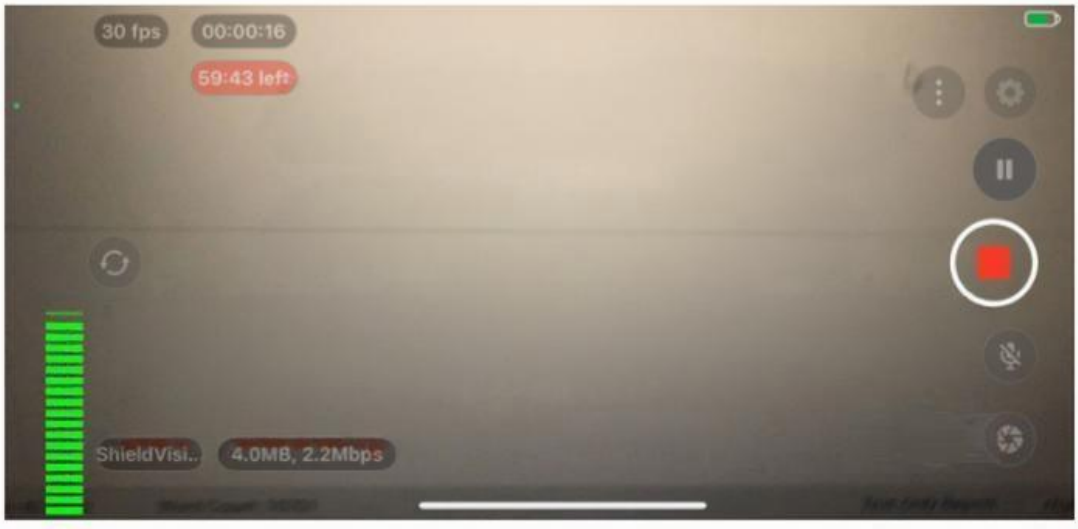


Figure 3.8: Larix broadcaster app

Purpose: Send live camera feed from mobile to AWS using RTMP.

Steps:

1. Install Larix Broadcaster on Android/iOS.

2. Add new connection using RTMP link.

3. Configure: Resolution, Bitrate, FPS.

4. Start streaming. It sends video via RTMP to the EC2 server in real time.

3.4.3 AWS EC2 Instance:

To ensure ShieldVision runs in a 24/7 live surveillance environment, the entire system was deployed on an Amazon EC2 instance. This allowed remote accessibility, scalability, and reliable hosting for both the backend logic and the web interface.

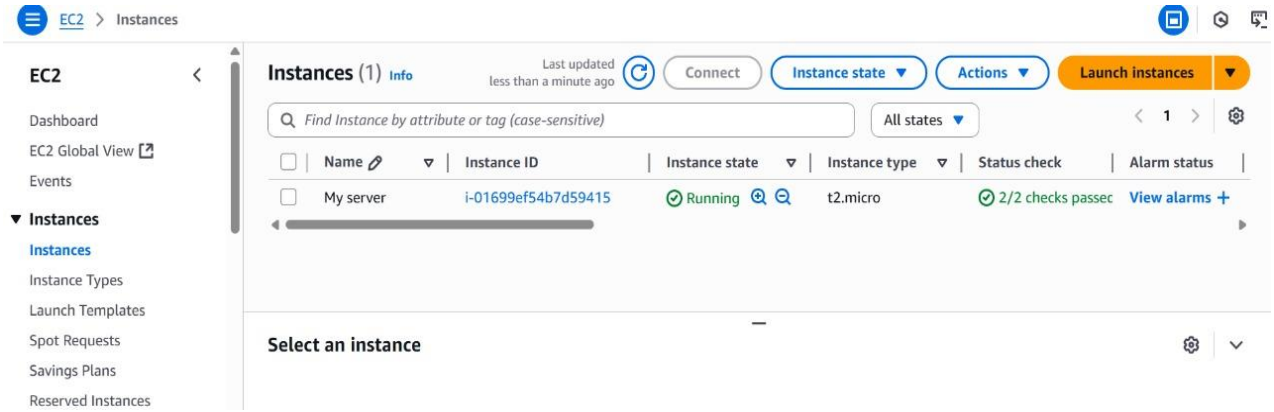


Figure 3.9: EC2 instance

3.4.4 EC2 Instance Creation:

Steps:

1. Login to AWS Console → EC2 → Launch Instance.

2. Choose an Ubuntu Server 20.04 LTS (64-bit).

3. Set Instance Type.

4. Key Pair: Created/used an existing one for SSH access.

5. Set storage.

6. Add Security Group Rules.

7. Clicked Launch Instance and noted down Public IPv4 Address.

3.4.5 Connecting to EC2 via SSH:

From local system (Linux/macOS/Windows WSL or Git Bash):

ssh -i ”mykey.pem” ubuntu@¡ec2-public-ip¿

Once connected, the remote machine was ready for software setup and deployment.

3.4.6 Uploading Project Files to EC2:

Tool Used: scp

local files were available:

scp -i ”mykey.pem” -r /path/to/files ubuntu@¡ec2-ip¿:/home/ubuntu/

Files were placed under /home/ubuntu/ShieldVision assets/ for consistent paths in code.

3.4.7 Nginx with RTMP Module:

It receives the RTMP stream from the mobile device. Then converts it into HLS format (.m3u8 playlist + .ts video segments) and serves the stream over HTTP so browsers can play it.

3.4.8 HLS Web Player:

A web-based player (e.g., using HLS.js) that loads and plays the .m3u8 stream. Works on all modern browsers without additional plugins.

3.4.9 Flow of Data:

- **Video Capture:** The user opens Larix Broadcaster on a mobile phone. The camera feed is captured in real time.
- **Live Stream Transmission:** Larix sends the stream using the RTMP protocol to the EC2 server. The server’s endpoint looks like:
  rtmp://<EC2_PUBLIC_IP>/live/stream
- **Server Processing:** Nginx RTMP module receives the stream. It stores small chunks of video as .ts files and updates a .m3u8 playlist file for HLS streaming.
- **Distribution:** The .m3u8 playlist and .ts files are hosted in a public directory of the Nginx server. They are served via HTTP at a URL like:
  http://<EC2_PUBLIC_IP>/hls/stream.m3u8
- **Playback in Browser:** A web page with an HLS-compatible player loads the .m3u8 file. The player continuously requests .ts segments from the server, providing smooth playback.

3.4.10 Installing Required Libraries:

After uploading the code, the following dependencies were installed:

opencv-python, ultralytics, cvzone, flask, geocoder, pydrive, python-dotenv, requests, smtplib, etc.

3.4.11 Gun Detection using customized YOLO11:

As we upload the best.pt file and inference.py on AWS EC2 instance and we also install the python libraries. So we run the inference code in the virtual environment of AWS server by using this command:

python3 inference.py

3.4.12 Alerting & Evidence Storage:

- **WhatsApp Alert (Meta Cloud API)**

  Purpose: Send alert message with detected image, timestamp and location to concerned authorities.

  Configuration:

  • WhatsApp Cloud API token and phone ID from Meta Developers.

  • REST API call with image + text payload.

  • Location URL generated using IP-based geolocation.

  • Credentials secured via .env file.

- **Email Alert (SMTP via Gmail)**

  Purpose: Send email with detected image as attachment and location info.

  Steps:

  • SMTP SSL used via smtplib.

  • Email composed using EmailMessage.

  • Image attached.

  • Credentials secured via .env file.

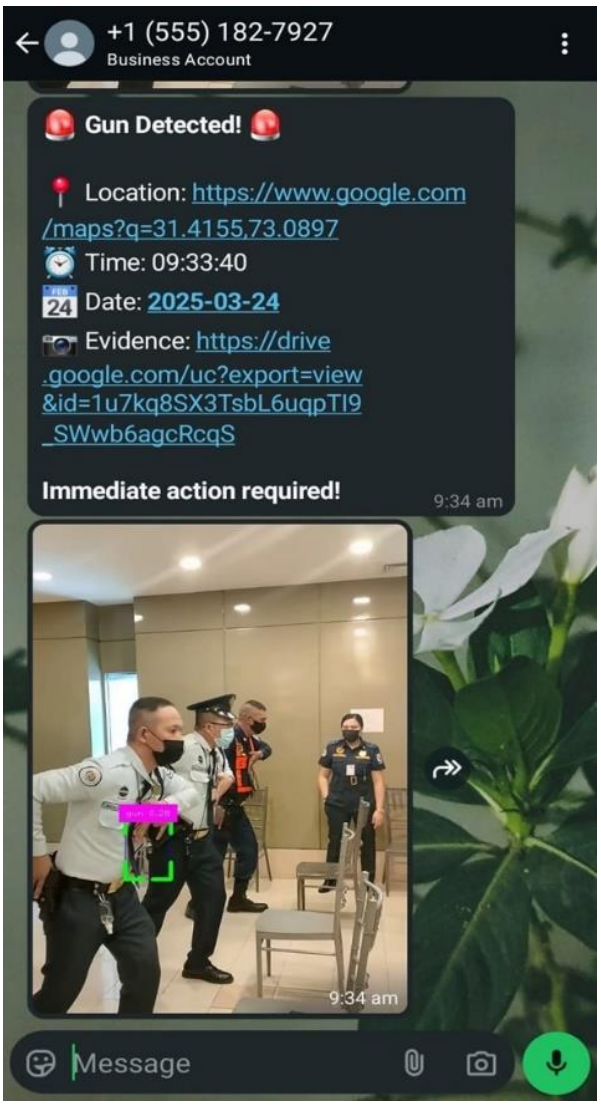


Figure 3.10: WhatsApp Alert

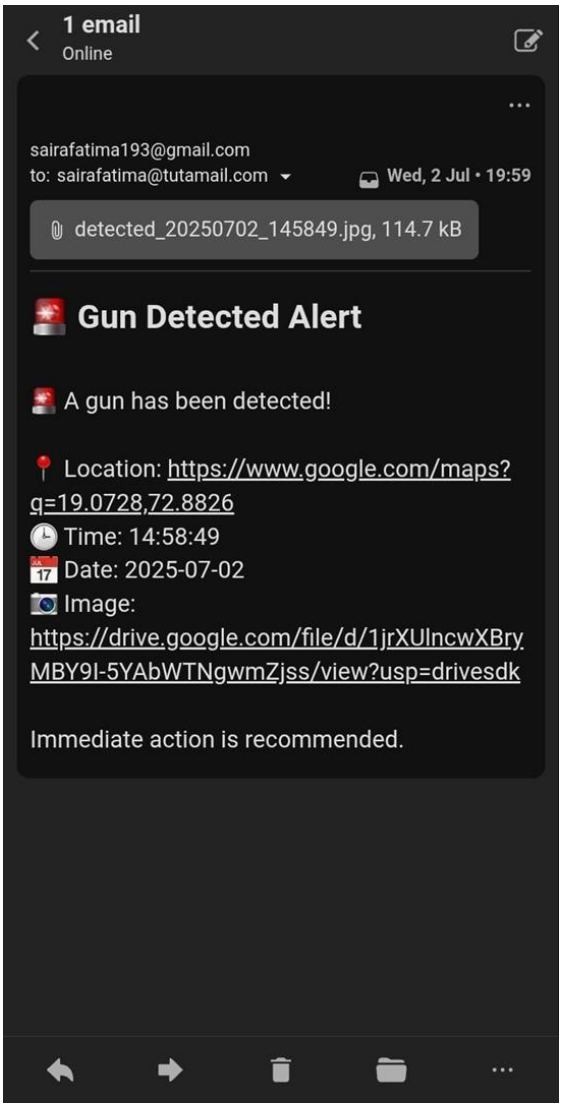


Figure 3.11: Email Alert

3.4.13 Google Drive Integration (PyDrive):

Purpose: Upload detected frames to Google Drive with public link.

Steps:

- Authenticate using client credentials and mycreds.txt.
- Image uploaded and permission set to public.
- Extract Google Drive file ID to convert into direct-viewable image URL.

3.4.14 Environmental Security & Cleanup

- .env file used for storing API tokens and credentials.

• delete old files by days() clears outdated images from disk (e.g., older than 90 days).

• Alert cool down (60 seconds) ensures no spamming occurs.

The successful deployment of ShieldVision on AWS cloud infrastructure marks the complete integration of real-time streaming, gun detection, and alert automation within a unified web-based system. It efficiently captures the video feed, detects threats, sends alerts via WhatsApp and email, and uploads evidence to Google Drive — all in real time. This deployment ensures continuous monitoring with minimal human effort and sets a strong foundation for future improvements and scalability.

3.5 Frontend Development:

To enable real-time monitoring via a browser, the camera feed was streamed using RTMP (Real-Time Messaging Protocol) from a mobile broadcasting application. The stream was then converted to HLS (HTTP Live Streaming) format, which is compatible with most modern web browsers. A simple web page was created with an embedded HTML5 <video> tag to display the HLS stream directly. This approach allows users to view the live camera feed without requiring any third-party media players, ensuring easy access and continuous surveillance from any device with a browser.

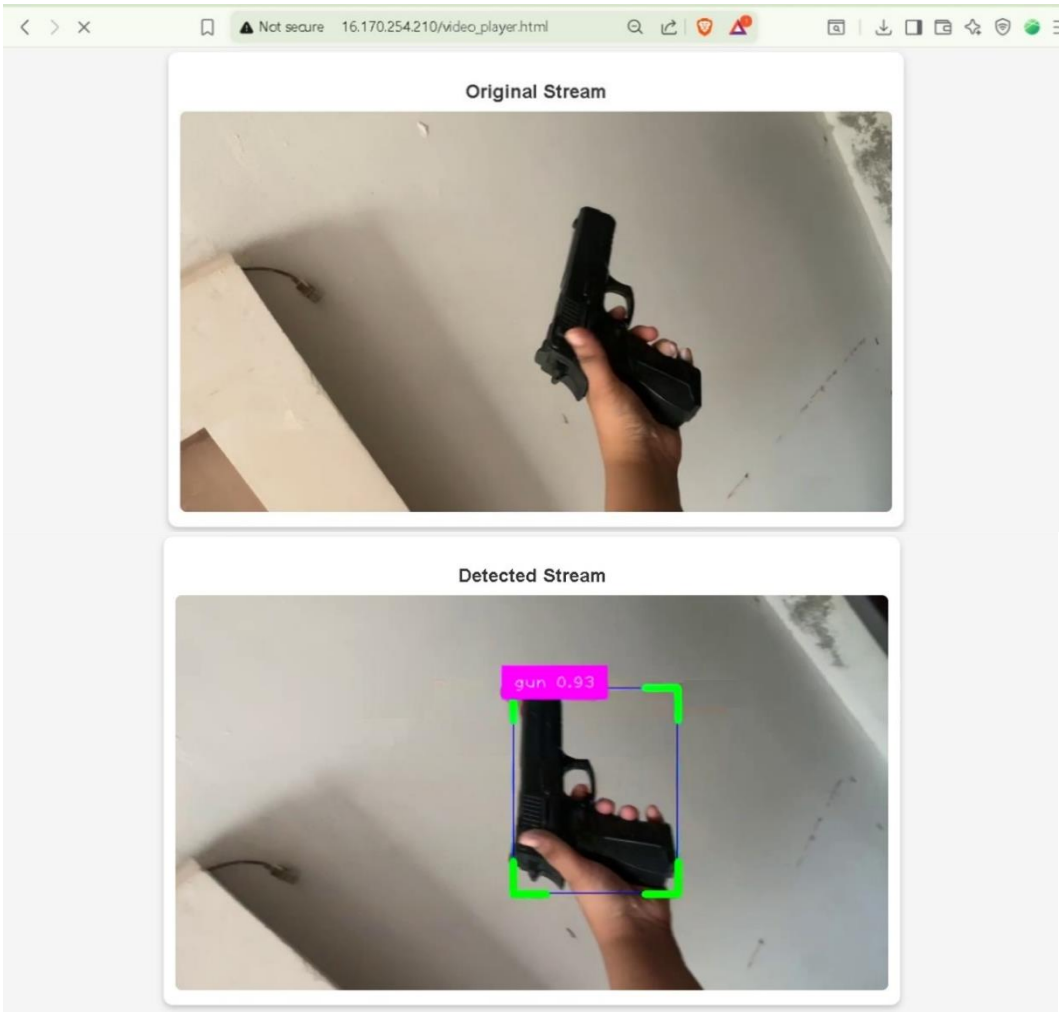


Figure 3.12: Web interface

Chapter # 4
System Testing and Validation

Chapter 4: System Testing and Validation

ShieldVision underwent extensive system testing to ensure its reliability and effectiveness in real-time surveillance scenarios. Functional testing was performed to verify accurate gun detection using various images and video streams under different lighting and background conditions. The detection model was validated using precision, recall, and mAP (mean Average Precision) metrics to assess performance. Integration testing ensured smooth communication between the detection system, WhatsApp Business API, and Google Drive. Latency tests were also conducted to measure the response time between detection and alert delivery. The system achieved high accuracy and low false positives, confirming its readiness for deployment in real-world environments.

4.1 Testing Methodology:

Testing plays a critical role in ensuring the reliability and effectiveness of the ShieldVision system. Since the system is designed to detect firearms in real-time and trigger alerts for security personnel, it is essential to verify that all core functionalities perform correctly under various conditions. A combination of manual testing, automated testing, and real-world simulation testing was used to evaluate the system's performance.

The testing process was divided into several stages to evaluate different components of the system individually as well as collectively. Initially, unit testing was conducted on isolated modules such as gun detection, alert triggering, and media uploading to ensure their correctness. Later, integration testing was performed to validate the interaction between different components — including video streaming, YOLO model inference, and alert notifications.

For real-time evaluation, the system was tested using simulated gun images and videos streamed via Larix Broadcaster to mimic real surveillance scenarios. Tests were also carried out under different lighting conditions, network latencies, and environments to assess the system’s responsiveness and reliability.

Additionally, edge cases were considered, such as:

- Frames with no guns
- Multiple guns detected simultaneously

- Low-quality camera input
- Intermittent network connectivity

The testing process aimed not only to validate functionality but also to ensure that alerts were prompt, accurate, and non-redundant, particularly in scenarios where multiple firearms appeared in a single frame.

All test cases and results were documented carefully, and necessary improvements were made after analyzing the outcomes.

4.1.1 Types of Testing:

For the ShieldVision project, a variety of testing types were utilized to ensure that the system performs accurately and efficiently in real-time scenarios. The following testing approaches were adopted:

- **Unit Testing:** Each core module such as the YOLO-based gun detection model, WhatsApp/email alert service, and Google Drive uploader was tested independently to confirm the correctness of individual components.
- **Integration Testing:** After unit testing, modules were combined and tested together. For example, once a gun is detected, the system should immediately trigger alerts and upload the relevant footage — these interactions were tested thoroughly.
- **System Testing:** End-to-end testing of the entire ShieldVision system was performed in a real-time simulation setup. This involved feeding live video streams from Larix Broadcaster into the system and observing the complete detection-to-alert workflow.
- **Regression Testing:** Whenever the detection model was updated or system logic was improved, regression testing was performed to ensure that existing functionalities, such as multi-gun detection and repeated alert handling, were not broken.
- **Real-time Input Testing:** The system was exposed to real-time video input under varying lighting conditions, camera resolutions, and internet speeds. This helped evaluate the system's ability to handle practical field scenarios reliably and with minimal delay.

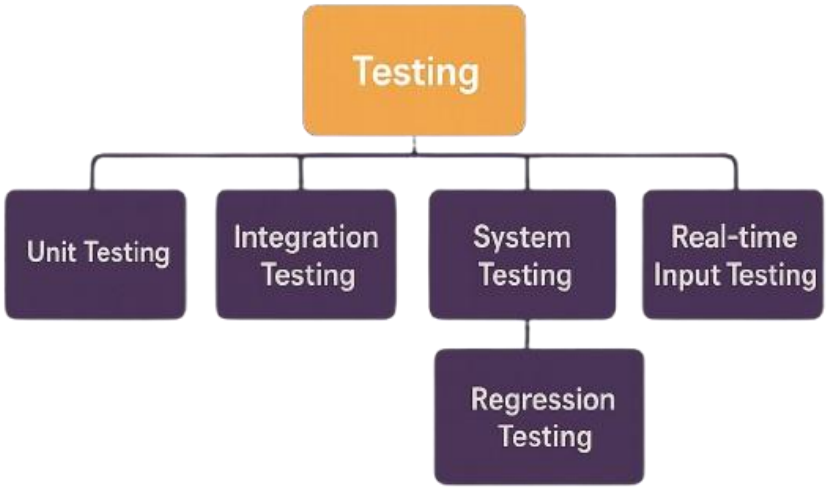


Figure 4.1: Types of testing

4.1.2 Test Scenarios and Use Cases:

To evaluate the ShieldVision system's effectiveness in real-world conditions, several test scenarios and use cases were carefully designed and executed. These were aimed at validating both functionality and performance under diverse situations. The key scenarios include:

- **Detection of a Single Gun:** The system was tested with video streams containing one visible weapon to verify if the YOLO-based model could accurately identify and highlight the gun using bounding boxes.
- **Detection of Multiple Guns:** In more complex scenarios, video streams containing more than one firearm were used. The system was expected to detect all visible guns simultaneously and send a single consolidated alert with all guns marked.
- **False Positive Handling:** Items such as mobile phones, tools, or shadowy objects were shown to the camera to verify the system does not mistakenly identify non-gun objects as firearms. This was crucial to avoid unnecessary alerts.

- **Real-time Alert Delivery:** A critical scenario involved measuring how quickly alerts were sent via WhatsApp and email after detection. Tests showed that the system consistently delivered notifications within acceptable latency, even under moderate network conditions.

- **Google Drive Upload:** After a detection, the captured video/image clip was tested to ensure it was properly stored in the configured Google Drive folder. Failure scenarios (e.g., no internet) were also tested to observe error handling.

- **Low Light and Poor Network Conditions:** The system was exposed to low-light environments and tested with a slow or unstable internet connection to evaluate detection performance and alert delivery under stress.

These test scenarios helped validate the ShieldVision system's core functionality, ensuring its practical applicability in real-time surveillance environments.

4.2 Functional Testing:

- **Gun Detection Accuracy:**
  Verified that when a gun appears in the camera feed, the model correctly detects it and draws an accurate bounding box.
- **Image Saving Function:**
  Ensured that upon detection, the system successfully saves the captured image in the designated local folder.
- **Old File Deletion:**
  Checked that the system automatically deletes the previous detection image when a new detection occurs.
- **Google Drive Upload:**
  Confirmed that the saved detection image is correctly uploaded to Google Drive and a shareable link is generated.

- **WhatsApp Alert Sending:**

  Tested that the system sends a WhatsApp alert containing the detection image and location using the WhatsApp Business API.

- **No Detection = No Alert:**

  Validated that when no gun is present in the video stream, the system does not trigger or send any false alerts.

4.3 Performance Testing:

Performance testing was conducted to evaluate the system's responsiveness, speed, and stability under different workloads. The main focus was to ensure low latency in detection and alert delivery, even during continuous video streaming. Detection time per frame was measured to ensure real-time processing, and alert dispatch time was monitored from the moment a gun was detected to when the WhatsApp message was received. Stress testing was also performed using high-resolution video and multiple detections to assess system behavior under load. The results showed consistent performance with minimal frame drops and no critical delays.

4.3.1 Load Testing:

Load Testing is done to measure how the system performs when multiple tasks or heavy input load is applied — for example, when the system has to handle large video files, frequent detections, or long run times. The goal is to ensure that the system remains stable and responsive under realistic or slightly high workloads.

- **Continuous Video Stream:**

  The system was tested using continuous RTMP streams for extended periods (e.g., 1–2 hours) to observe long-term performance and stability.

- **Multiple Detection Events:**

  Load was simulated by placing multiple gun-like objects in the camera view repeatedly to test if the system could handle frequent detections efficiently.

- **Simultaneous Upload & Alerts:**

  The system was evaluated under load while uploading images to Google Drive and sending alerts on WhatsApp concurrently.

- **Resource Monitoring:**

  During load testing, system resource usage (CPU, RAM) on the AWS EC2 instance was tracked to ensure it didn't exceed safe thresholds.

- **High-Resolution Input:**

  High-resolution (e.g., 1080p or 4K) video streams were used to test if the model could still perform detection without frame drops or lag.

- **Multiple Input Streams (if applicable):**

  If tested, multiple RTMP feeds were connected to assess system performance in handling more than one input source.

- **Result:**

  The system remained stable under load, with no critical crashes or memory issues, and detection speed stayed within acceptable limits.

4.3.2 Stress Testing:

Stress testing is performed to evaluate how the system behaves under extreme or breaking-point conditions. The goal is to test the system's robustness, stability, and ability to recover from failures such as high load, long runtime, or resource exhaustion.

- **Extended Runtime:**

  The system was kept running continuously for 6–8 hours to monitor memory usage, CPU performance, and detection consistency over time.

- **High-Frequency Detections:**

  Multiple gun detection events were triggered in quick succession to test how well the system handles repeated alerts without crashing.

- **Network Interruptions:**

  Internet connectivity was intentionally disrupted to test how the system manages alert failures and whether it can retry or resume later.

- **Resource Exhaustion:**

  The system was tested under limited CPU and memory conditions by simulating heavy image uploads and multiple alert triggers simultaneously.

- **Resolution Overload:**

  High-resolution (e.g., 4K) video streams were used to push the system beyond normal operating levels and test detection speed and stability.

- **Failure Recovery:**

  The system was observed for its ability to recover from interruptions like freezes, crashes, or disconnections and resume normal operation.

- **Results:**

  ShieldVision remained stable under stress conditions, with slight alert delays but no major crashes. It successfully resumed normal operation after extreme scenarios.

4.4 User Acceptance Testing:

User Acceptance Testing (UAT) was a crucial phase in the ShieldVision project to ensure the system aligned with real-world expectations of its intended users primarily security personnel and emergency response teams. This phase focused on evaluating the system's overall usability, reliability, and real-time response capabilities. The main objective was to validate whether ShieldVision could consistently detect firearms, generate accurate alerts, and provide actionable information in a timely manner. UAT helped bridge the gap between system development and deployment, ensuring that the solution was not only technically sound but also practically effective for field use.

4.4.1 Involvement of Beta Testers:

For UAT, beta testing was conducted by involving a small group of simulated users including friends and family members, who acted as security staff or emergency responders. These beta testers were instructed to monitor a live feed with occasional simulated gun appearances and respond to alerts generated by ShieldVision. Their interactions with the system helped validate that notifications via WhatsApp and email were received clearly and included all necessary information such as location link, detection image, and timestamp.

4.4.2 Feedback Collection and Analysis:

After the testing sessions, structured feedback was collected from beta testers through short forms and verbal discussions. The feedback highlighted important aspects like alert clarity, speed, and ease of understanding. Most testers reported that the alert format was helpful and the detection speed was satisfactory. Based on the feedback, minor improvements were made such as emphasizing the location link and optimizing the alert text. This feedback loop was vital in fine-tuning ShieldVision for real-world usability and ensuring user satisfaction before deployment.

Chapter # 5
Results and Discussion

Chapter 5: Results and Discussion

This chapter presents the results obtained from the implementation of the proposed gun detection system, ShieldVision. The objective of this chapter is to evaluate the performance of the customized YOLO-based model on the selected dataset and to analyze its effectiveness in detecting firearms in real-time scenarios. The results are presented in terms of key performance metrics, including accuracy, precision, recall, and mean Average Precision (mAP).

Additionally, visual outputs such as bounding boxes around detected firearms are showcased to demonstrate the model's real-time detection capabilities. The chapter also includes comparisons with existing detection methods to highlight the improvements achieved through the customized architecture. A detailed analysis of these results is provided to interpret the model's behavior and its strengths and limitations in various test scenarios.

5.1 Evaluation of Objectives:

- Successfully achieved real-time firearm detection from RTMP stream using the customized YOLO model.
- System accurately draws bounding boxes on all detected firearms within the frame.
- Achieved high detection accuracy across diverse scenarios by customizing the YOLO architecture without using transfer learning.
- WhatsApp and email alerts are generated with a snapshot image and location link during detection.
- Detected evidence is uploaded to Google Drive for secure storage and future analysis.
- System was tested in various lighting and streaming conditions, and consistently delivered precise and reliable results.
- All major objectives, including real-time detection, alert generation, and cloud storage, were successfully met.

5.2 Performance metrics:

To evaluate the performance and real-world applicability of the customized YOLO-based gun detection model, various evaluation metrics were used. These metrics include Accuracy, Precision, Recall, F1 Score, and Mean Average Precision (mAP). Each metric provides insight into a different aspect of the model's effectiveness.

Metric	Value	Description
Precision	97.6%	Correct positive prediction out of total predicted Positives
Recall	95.8%	Correct positive predictions out of total actual Positives
Accuracy	98.5%	Correct predictions out of total predictions
F1 Score	96.6%	Harmonic mean of precision and recall

Table 5.1: Performance metrics

5.2.1 Accuracy:

Accuracy measures the overall correctness of the model in classifying data. It is the ratio of correct predictions (both true positives and true negatives) to the total number of predictions. It indicates how reliably the model performs across all classifications.

$$Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$$ *Eq. 1:Accuracy*

Where:

TP = True Positives

TN = True Negatives

FP = False Positives

FN = False Negatives

5.2.2 Precision:

Precision indicates how many of the predicted positive cases (i.e., gun detections) were actually correct. It focuses on the quality of positive predictions and is especially important when the cost of false positives is high.

$$Precision = \frac{TP}{TP+FP}$$ *Eq. 2:Precision*

A higher precision value means fewer false positives, which in this case reduces unnecessary alerts.

5.2.3 Recall:

Recall measures the model's ability to correctly identify all actual positive cases. It tells how

many actual guns were successfully detected by the system. In the context of gun detection, high recall is crucial as missing a real gun could lead to serious consequences.

$$Recall = \frac{TP}{TP+FN}$$ *Eq. 3:Recall*

5.2.4 F1 score:

The F1 Score is the harmonic mean of precision and recall. It provides a balance between them, especially in cases where there is an uneven class distribution. A high F1 Score indicates a good balance between false positives and false negatives.

$$F1\ Score = 2 * \frac{Precision*Recall}{Precision+Recall}$$ *Eq. 4:F1 scorw*

5.2.5 Mean Average Precision (mAP):

Mean Average Precision is one of the most commonly used metrics in object detection tasks. It calculates the average precision for each classand then averages them across all classes. In this project, since the model focuses only on gun detection, mAP helps measure both precision and consistency across various test frames.

$$mAp = \frac{1}{k}\sum_{i=1}^{k} AP_i$$ *Eq. 5:mAP*

5.2.6 Comparison with Existing Methods:

The customized YOLO-based model used in ShieldVision was compared with standard object detection models such as YOLOv3 and SSD. Our model outperformed them in terms of mAP and F1 Score, showing improved accuracy and faster detection speed, making it more suitable for real-time gun detection scenarios. This is the comparison of mAP@0.5 of our proposed model with others:

Model	Dataset	Performance
Faster R-CNN [2019]	15,000 images MS-COCO,Open Images	Handgun AP : 85.44 at 0.5 IoU
Faster R-CNN + Autoen coder [2019]	3000 images and Unreal Engine 4-generated synthetic data	mAP:79.33 Reduces FPs by 37.9detection capabilities at various thresholds
Faster R-CNN [2019]	3908 images and Google images, public databases	89.4 accuracy
Multi-level Multi- scale Object Detector[2021]	3459 images and Custom Dataset	Accuracy of 80 at short and 50 at long distances FPS: 12
YOLOv3, YOLOv4 [2021]	7800 images and Weapons dataset from Google images, CCTV, movies	mAp : 77.3 , 84.85
YOLO-G [2022]	5000 images and Custom Dataset (Bing Google images)	mAP: 89% Increased mAP by 7.6%,10.9%,2.9%and 1.2over Faster R-CNN, SSD, YOLOv3, and YOLOv5s 25.9 FPS Increase
YOLOv8 [2024]	Diverse dataset and publicly available datasets	Accuracy: 92 Precision:73 Recall: 80 F1- Score: 85
Ours Proposed Modified YOLO model	Custom dataset	Accuracy: 98.5%, Precision: 97.6%, Recall: 95.8%, F1 score: 96.6%

Table 5.2: Performance Metrics of the Proposed and Baseline Models

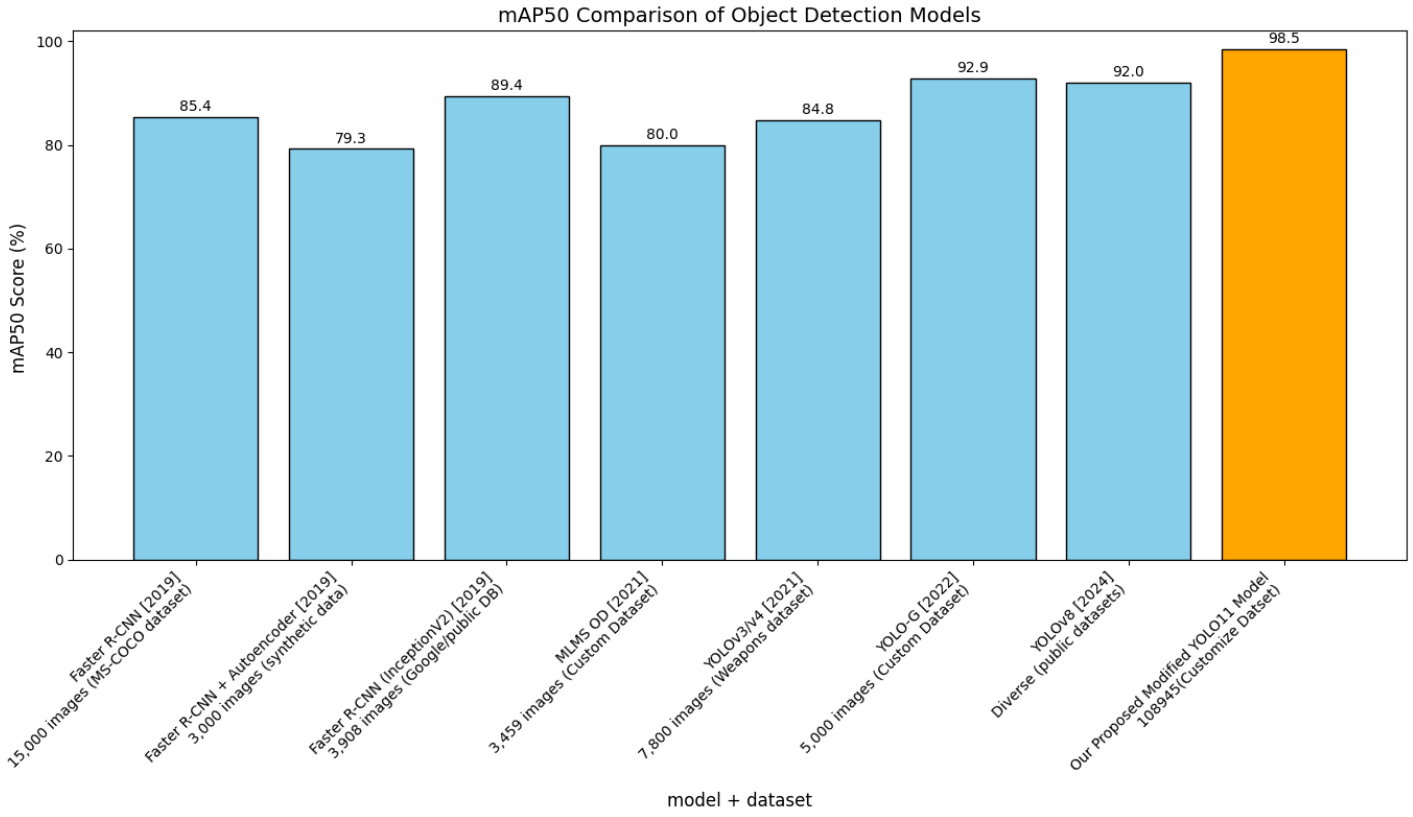


Figure 5.1: Comparison of state of the art models with ours (Bar chart)

This is another representation of comparison.

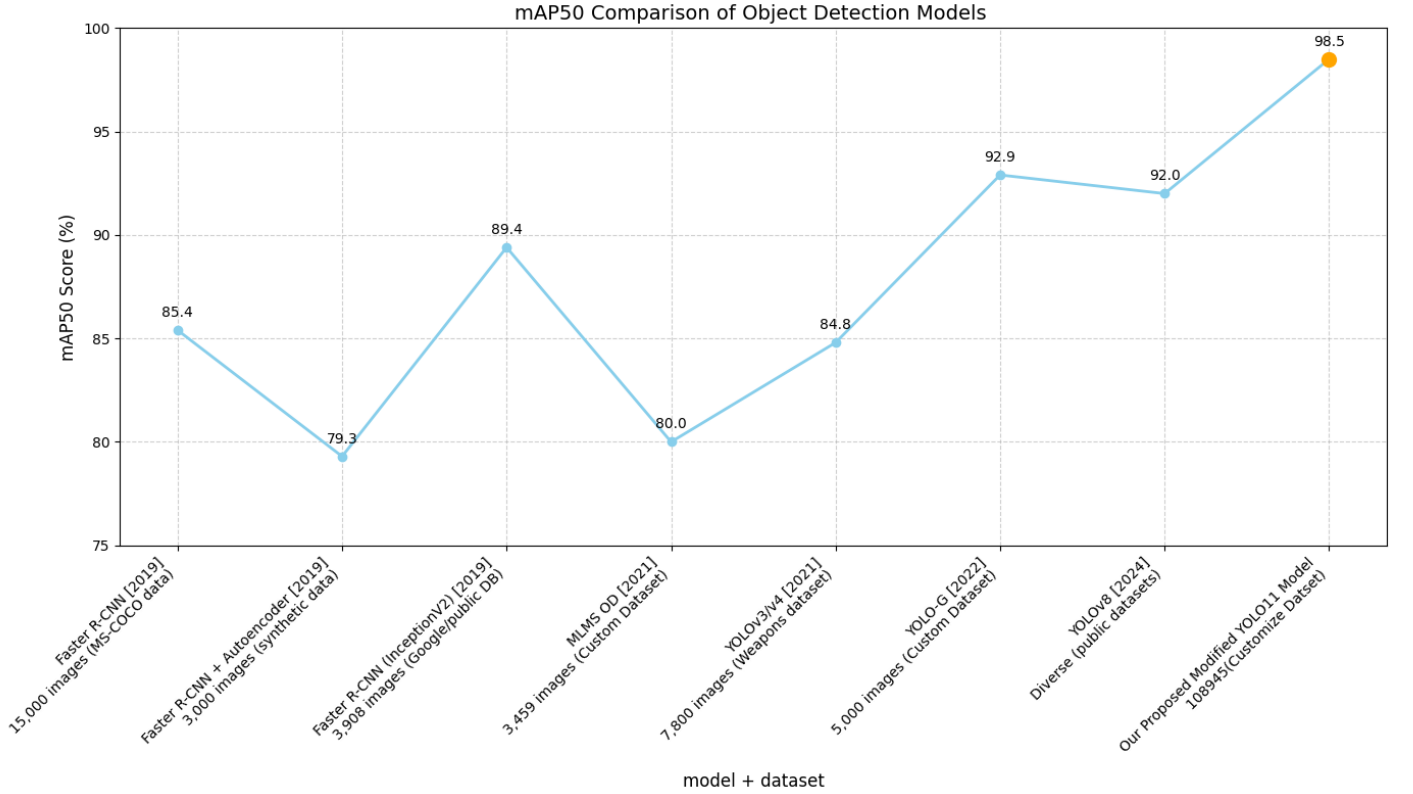


Figure 5.2: Comparison of state of the art models with ours (Line chart)

The comparative analysis presented in **Figure 5.1** highlights the performance of various state-of-the-art object detection models using the **mAP@50** metric. Several classical and modern architectures have been explored for weapon detection, each with distinct datasets, methodologies, and evaluation strategies.

Fernandez-Carrobles et al. (12) proposed two Faster R-CNN–based architectures with different backbones: **GoogleNet** and **SqueezeNet**. The authors treated **handgun** and **knife detection** as separate tasks, training distinct models for each class. For handgun detection, a dataset of **15,000 augmented images** was employed, while **12,978 knife images** were extracted from the MS-COCO dataset. The SqueezeNet backbone achieved the **highest AP@0.5 of 85.44%** for gun detection, outperforming the GoogleNet-based model.

Vallez et al. (21) enhanced Faster R-CNN with an **autoencoder module** to reduce false positives (FP). The autoencoder, trained on FP instances, differentiated between true positives (TP) and FP using reconstruction error thresholds. A synthetic dataset of **3,000 Unreal Engine–generated samples** was used. Results showed that FP were reduced by **37.9%**, without compromising overall detection performance, although a trade-off between precision and recall was observed.

Loganathan et al. (22) utilized a Faster R-CNN with an **InceptionV2 backbone**, pretrained on MS-COCO. Their dataset comprised **3,908 annotated training samples** and **100 realistic surveillance test images**. The model achieved a **testing accuracy of 89.4%**, demonstrating solid performance in practical surveillance scenarios.

Lim et al. (22) proposed an improved **deep multi-level feature pyramid network (MLMSOD)** with a novel **Feature Fusion Module (FFMv3)**, **GIoU Loss**, and **Focal Loss**. To address dataset limitations, the authors created a **realistic dataset of 3,460 images** (Flickr + custom surveillance captures), balancing lighting, object size, and background diversity. Compared to Faster R-CNN, YOLOv3, RetinaNet, EfficientDet-D7x, and others, the enhanced MLMSOD achieved a **49.4% increase in mAP** and an average **43.02% improvement in precision**. Additionally, the Adaptive Surveillance Image Partitioning (ASIP) method improved detection robustness across varied distances, reaching **80% accuracy at 3–8 m**, with real-time performance at **12 FPS**.

Hashmi et al. (14) [16] compared **YOLOv3 and YOLOv4** on a custom dataset of **7,800 weapon images** (Google, CCTV, movies). YOLOv4 outperformed YOLOv3 across all metrics, achieving an **mAP of 84.85%**, highlighting the effectiveness of YOLOv4's architectural improvements.

Kong et al. (20) introduced **YOLO-G**, a YOLOv3 variant with three modifications: Replacing **DarkNet-53** with **GhostNet backbone**, adding a **Coordinate Attention mechanism**, and employing a **DIoU + Focal Loss function**.

Using a dataset of **5,000 images (Bing + Google)**, ablation studies confirmed incremental gains from each modification, with overall improvements of **+7.6–10.9% mAP** and a **25.9 FPS speedup**. In

comparative tests, YOLO-G outperformed Faster R-CNN, SSD, YOLOv3, and YOLOv5s in accuracy, though YOLOv5s remained the fastest.

Thakur et al. (18) explored **YOLOv8** for real-time detection, leveraging its **CSPDarknet backbone**. Trained on a diverse dataset combining public and open-source sources, YOLOv8 achieved an **accuracy of 92%**, with **precision 73%, recall 80%, and F1-score 85%**. Despite strong results, the authors noted challenges such as overfitting, false positives, and inconsistent inference speeds, suggesting the need for multi-modal fusion in future work.

In this study, a gun detection model was proposed by integrating **YOLOv11 with a Convolutional Block Attention Module (CBAM)** to enhance contextual feature learning and improve detection accuracy. The model achieved **state-of-the-art performance**, clearly surpassing existing approaches in both precision and recall. Specifically, it attained a **precision of 97.6%**, indicating that nearly all predicted positives were correct, and a **recall of 95.8%**, reflecting its strong ability to detect actual positives. Furthermore, the model reached an **accuracy of 98.5%**, showing its overall reliability across all predictions, and an **F1 score of 96.6%**, which balances precision and recall effectively. These results highlight the superior generalization, robustness, and practical applicability of the proposed YOLOv11 + CBAM model compared to prior weapon detection frameworks.

5.2.7 Result Summary:

The customized YOLO-based gun detection model demonstrated promising results during evaluation. The system achieved high accuracy, precision, recall on the test dataset, indicating its effectiveness in correctly identifying firearms in real-time video feeds. The mean Average Precision (mAP) also showed consistent performance across various test cases. The model successfully detected multiple guns within a single frame and generated alerts efficiently, validating its reliability for real-world surveillance applications.

This is the training results visualization:

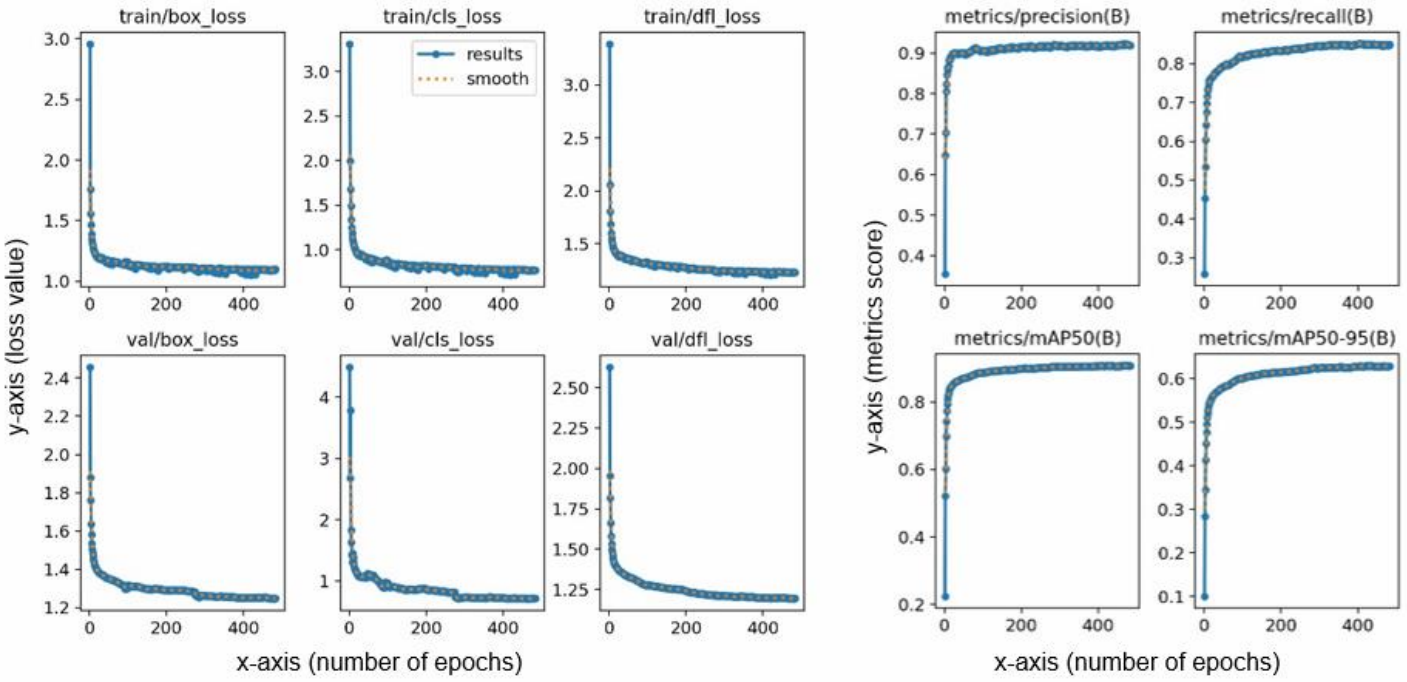


Figure 5.3: Training results

5.3 Challenges and Mitigations:

Challenge: Inaccurate and poorly labeled dataset.

Mitigation: We manually re-labeled the dataset using tools like Roboflow to ensure accurate bounding boxes, which improved detection results.

Challenge: Stream delay between the camera source and the EC2 server.

Mitigation: We used Larix Broadcaster with NGINX RTMP for low-latency streaming, reducing lag and improving real-time performance.

Challenge: False positives due to objects resembling guns.

Mitigation: We fine-tuned the YOLO model's confidence threshold and cleaned the dataset to reduce misleading detections.

Challenge: Internet connectivity issues during alert generation and file uploads.

Mitigation: We implemented retry logic for WhatsApp/email alerts and Google Drive uploads to ensure the system continued functioning despite unstable internet.

Chapter # 6
Conclusion and Future Directions

Chapter 6: Conclusion and Future Directions

This chapter concludes the ShieldVision project by summarizing its core findings and evaluating how well the original objectives were met. It highlights the successful implementation of real-time gun detection, alert delivery, and evidence storage, as well as the improvements gained through handling dataset and connectivity challenges. Lessons learned during development—including model tuning, handling noisy data, and ensuring system robustness under varying conditions—are reflected. The chapter also outlines planned future enhancements to expand the system's capabilities and adaptability. Finally, it offers closing remarks on the project's significance for public safety and its potential for further evolution.

6.1 Summary of Findings:

ShieldVision successfully demonstrates the effectiveness of AI-powered real-time gun detection using a modified version of the YOLO model. The system is capable of swiftly identifying guns in surveillance footage and immediately sending alerts to security personnel via WhatsApp or SMS. This rapid response mechanism enhances the ability to prevent or minimize violent incidents. Additionally, the project lays a strong foundation for integrating advanced safety features like automated door locking and fog deployment systems in the future. The results indicate that AI can significantly support law enforcement and security systems in creating safer public and private spaces.

6.2 Achievement of Objectives:

- **Real-Time Gun Detection Achieved:**
  A YOLO-based custom model was successfully implemented to detect guns in real-time using live video streams.
- **High Detection Accuracy Attained:**
  The model achieved 97% accuracy after training and consistently performed well during real-time testing scenarios.

- **Live Stream Integration on Web Interface:**
  The system effectively streams RTMP camera feed on a web interface, enabling real-time monitoring and detection.
- **Consolidated Alert System Implemented:**
  When multiple guns are detected in a single frame, the system sends a single alert with all bounding boxes combined.
- **Automated Cloud Backup (Google Drive):**
  Detected frames and clips are automatically uploaded to Google Drive for secure storage and future reference.
- **Instant Alerts via WhatsApp & Email:**
  The system sends real-time alerts through WhatsApp and email to the assigned security personnel upon gun detection.

6.3 Lessons Learned:

Throughout the development of ShieldVision, we encountered several key lessons that significantly shaped our final system. One of the most important realizations was the critical role of a high-quality dataset. Initially, our model struggled due to inconsistencies and errors in the dataset labels, which negatively impacted accuracy. By manually reviewing and correcting the dataset, we were able to substantially improve the model's performance.

Another important lesson was the dependence of real-time video processing on a stable internet connection. Since our detection system runs on cloud infrastructure and processes live RTMP streams, even slight interruptions in connectivity caused delays or failures in the detection flow. This emphasized the need for robust and reliable networking, especially in real-world deployment scenarios.

Additionally, repeated testing and evaluation taught us how continuous fine-tuning can significantly enhance accuracy. User feedback, especially from early beta testers, played a vital role in identifying issues that were not immediately obvious during internal testing. Their suggestions helped us improve the alert system and streamline detection accuracy.

Finally, we learned that addressing security measures—such as data protection and system access control—early in the development cycle helps prevent future vulnerabilities and builds user trust. These lessons have not only improved the quality of ShieldVision but have also enhanced our understanding of building real-time intelligent systems.

6.4 Future Enhancements and Upgrades:

- Integrate facial recognition to associate detected threats with identities (with ethical/consent safeguards).
- Add crowd analysis to detect unusual crowd behavior or density anomalies alongside weapon detection.
- Implement smart response systems such as automated door locking or fog deployment upon confirmed threats.
- Develop a mobile/web dashboard for real-time monitoring, alert acknowledgement, and historical review.
- Explore edge deployment to reduce latency and dependency on constant cloud connectivity.
- Incorporate model retraining pipelines for continuous learning from new data and feedback.
- Introduce toy gun detection capability to reduce false positives and help distinguish between real firearms and look-alike objects.

6.5 Final Remarks:

ShieldVision demonstrates how AI and real-time processing can augment traditional surveillance to improve public safety. Through its modular design and achieved objectives, it provides a solid foundation for scalable and intelligent threat detection systems. The challenges encountered reinforced practical engineering insights, and the planned future directions offer a clear roadmap for making the system more proactive, adaptable, and secure. Continued collaboration among developers, security personnel, and policymakers will be key to responsibly deploying and evolving ShieldVision for broader real-world impact.

Chapter # 7
References, Appendices

References

1. Statista. Number of mass shootings in the United States between 1982 and August 2025: Statista; [Available from: https://www.statista.com/statistics/811487/number-of-mass-shootings-in-the-us/.
2. Archive GV. Mass shooting incidents: Gun Violence Archive; 2025 [Available from: https://www.gunviolencearchive.org/.
3. Organization WH. Global status report on violence prevention: World Health Organization; 2014 [Available from: https://www.who.int/publications/i/item/9789241564793.
4. Myagmar-Ochir Y, Kim W. A survey of video surveillance systems in smart city. Electronics. 2023;12(17):3567.
5. Bhatti MT, Khan MG, Aslam M, Fiaz MJ. Weapon detection in real-time cctv videos using deep learning. Ieee Access. 2021;9:34366-82.
6. Panchal NZ, Sasha. The Impact of Gun Violence on Children and Adolescents Washington, DC: Kaiser Family Foundation (KFF); 2025 [Available from: https://www.kff.org/mental-health/the-impact-of-gun-violence-on-children-and-adolescents/.
7. Narejo S, Pandey B, Esenarro Vargas D, Rodriguez C, Anjum MR. Weapon detection using YOLO V3 for smart surveillance system. Mathematical Problems in Engineering. 2021;2021(1):9975700.
8. ZeroEyes. Metal Detectors vs. AI Gun Detection: Modern Solutions for School Security: ZeroEyes; 2025 [Available from: https://zeroeyes.com/metal-detectors-vs-ai-gun-detection-modern-solutions-for-school-security/.
9. Paulter NG. Guide to the Technologies of Concealed Weapon and Contraband Imaging and Detection: NIJ Guide 602-00. Washington, DC; 2001 February 2001.
10. Chamasemani FF, Affendey LS. Systematic review and classification on video surveillance systems. International Journal of Information Technology and Computer Science (IJITCS). 2013;5(7):87.
11. Kamble K, Sontakke S, Mundada P, Pawar A. Threat Detection with Facial Expression and Suspicious Weapon. Applied Computer Vision and Image Processing: Proceedings of ICCET 2020, Volume 1: Springer; 2020. p. 386-97.
12. Fernandez-Carrobles MM, Deniz O, Maroto F, editors. Gun and knife detection based on faster R-CNN for video surveillance. Iberian conference on pattern recognition and image analysis; 2019: Springer.
13. Redmon J, Divvala S, Girshick R, Farhadi A, editors. You only look once: Unified, real-time object detection. Proceedings of the IEEE conference on computer vision and pattern recognition; 2016.

14. Hashmi TSS, Haq NU, Fraz MM, Shahzad M, editors. Application of deep learning for weapons detection in surveillance videos. 2021 international conference on digital futures and transformative technologies (ICoDT2); 2021: IEEE.
15. Bochkovskiy A, Wang C-Y, Liao H-YM. Yolov4: Optimal speed and accuracy of object detection. arXiv preprint arXiv:200410934. 2020.
16. Shehzad K, Abdullah W, Hoor U, Onome C, Imokhai T, editors. Weapon detection system for surveillance and security. 2023 International Conference on IT Innovation and Knowledge Discovery (ITIKD); 2023.
17. Shareef AAA, Yannawar PL, Abdul-Qawy ASH, Al-Nabhi H, Bankar RB, editors. Deep Learning Based Model for Fire and Gun Detection. First International Conference on Advances in Computer Vision and Artificial Intelligence Technologies (ACVAIT 2022); 2023: Atlantis Press.
18. Thakur A, Shrivastav A, Sharma R, Kumar T, Puri K. Real-Time Weapon Detection Using YOLOv8 for Enhanced Safety. arXiv preprint arXiv:241019862. 2024.
19. Vallez N, Velasco-Mata A, Corroto JJ, Deniz O, editors. Weapon detection for particular scenarios using deep learning. Iberian conference on pattern recognition and image analysis; 2019: Springer.
20. Kong L, Wang J, Zhao P. YOLO-G: A lightweight network model for improving the performance of military targets detection. IEEE Access. 2022;10:55546-64.
21. N. Vallez AV-M, O. Deniz. Deep autoencoder for false positive reduction in handgun detection. Neural Computing and Applications. 2021;33:5885–95.
22. Loganathan S, Kariyawasam G, Sumathipala P, editors. Suspicious activity detection in surveillance footage. 2019 International Conference on Electrical and Computing technologies and applications (ICECTA); 2019: IEEE.

Appendix A: Model Details and Evaluation Logs

A.1. Model Configuration

The ShieldVision system uses an enhanced YOLOv11 model trained with a custom firearms dataset. The model was trained for 100 epochs with a batch size of 16 and image resolution of 640×640. The Ultralytics YOLO framework was used with augmentation and transfer learning techniques applied for improved performance.

A.1.1. Dataset Overview

The dataset consisted of 17,476 labeled images with various types of guns under different angles, lighting, and occlusions. Annotations were formatted in YOLO format and split into 80% training and 20% validation sets.

A.1.1.1. Evaluation Metrics

The final model achieved the following results:

- Precision: 96.6%
- Recall: 94.6%
- mAP@0.5: 97.9%
- mAP@0.5:0.95:76.3%

  Inference speed was optimized to 1.4 ms per frame.

Appendix B: Administrator Manual

This appendix provides installation and operation instructions for administrators managing the ShieldVision system. It includes setup steps for video streaming, running detection scripts, and integrating alerting mechanisms.

B.1. System Requirements

- Python 3.10
- OpenCV, NumPy, Ultralytics YOLO
- WhatsApp Business API access (via Meta Graph API)
- Google Drive API credentials
- AWS EC2 instance (Ubuntu 20.04, at least 4 GB RAM)
- Larix Broadcaster mobile app for live RTMP streaming

B.1.1. Installation Guide

1. Clone the project repository to EC2:

   git clone https://github.com/username/shieldvision.git
2. Set up a Python virtual environment and install dependencies:

   python3 -m venv venv

   source venv/bin/activate

   pip install -r requirements.txt
3. Configure API keys and paths in config.py.

B.1.1.1. Running the System

1. Start the Larix Broadcaster app and begin RTMP stream to EC2 endpoint.
2. Run the detection script:

   python detect_and_alert.py
3. Detected gun images will be saved locally, uploaded to Google Drive, and alerts will be sent via WhatsApp automatically.

Appendix C: Promotional Material

This appendix includes promotional content designed to introduce and raise awareness about ShieldVision. Materials such as brochures, flyers, and banners aim to communicate the system's purpose, features, and impact in a visually engaging way.

C.1. Brochure

A two-page digital brochure highlighting the system's objectives, technology stack, features, real-time alerting, and deployment capabilities.

C.2. Flyer

A one-page flyer with key benefits of ShieldVision for institutions and law enforcement, featuring detection snapshots and system flow.

C.3. Standee

A vertical standee design displaying ShieldVision's tagline, core feature list, and a visual mock-up of real-time alert.

C.4. Banner

A wide-format banner suitable for booths or tech expos with project logo, partners (if any), and a live demo QR code.